“Operating System Design and Implementation”

(Chapter 2)
Advance Process and Thread Management

Er. Bhesh Bahadur Thapa
bhesh.thapa@pu.edu.np

References

Textbook

- ✓ Operating System Concepts, 10th Edition, Abraham Silberschatz
- ✓ Modern Operating System, 3rd Edition, Andrew S. Tanenbaum
- ✓ Operating Systems Three Easy Pieces, R. A DUSSEAU
- ✓ Andrew S. Tanenbaum, Distributed Operating System

Contents

This Chapter will cover:

- Advanced Process Scheduling Techniques: Multi-core (Affinity based, Load Balancing, Cache and memory Optimization) and NUMA-aware scheduling
- Thread Libraries and Synchronization: Real-time (Semaphore-based and conditional variable-based synchronization protocols) and Adaptive synchronization techniques (Adaptive locks and lock free data structure-based synchronization protocol)
- Deadlock Detection Algorithms (Centralized, Distributed and Hierarchical)
- Distributed Mutual Exclusion Algorithms (Lamport's, Token and Non-Token Based Ricart Agarwala, Epidemic Algorithms, Maekawa's)
- Case Studies: Process Management in Linux and Windows.

Computer System Architecture

- A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common bus that provides access between components and shared memory

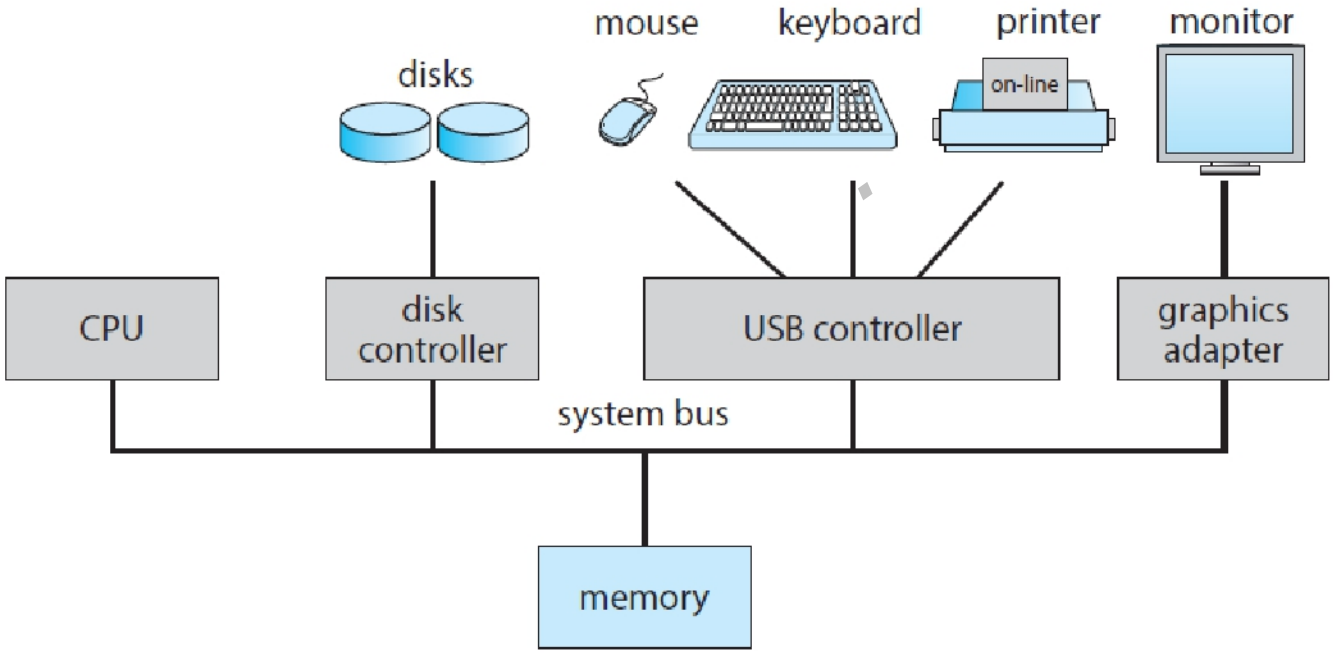


Figure 1.2 A typical PC computer system.

Computer System Architecture

- A computer system can be organized in a number of different ways, which we can categorize roughly according to the number of general-purpose processors used.

1. Single Processor System
2. Multi Processor System
3. Multi Core System
4. NUMA System

DEFINITIONS OF COMPUTER SYSTEM COMPONENTS

- CPU—The hardware that executes instructions.
- Processor—A physical chip that contains one or more CPUs.
- Core—The basic computation unit of the CPU.
- Multicore—Including multiple computing cores on the same CPU.
- Multiprocessor—Including multiple processors.

Computer System Architecture

Single Processor System

- Traditional computer systems primarily used a single processor with one CPU and a single processing core, which executes general-purpose instructions and stores data locally using registers.
- These systems also include device-specific processors, such as disk controllers, keyboard processors, and graphics controllers, which handle specialized tasks.
- Some special-purpose processors are managed by the operating system, which assigns tasks and monitors their status. For example, a disk controller uses its own queue and scheduling algorithm to reduce the main CPU's workload.
- A system is classified as a single-processor system if it has only one general-purpose CPU with a single processing core.

Computer System Architecture

Multi Processor System

- Modern computing, from mobile devices to servers, is dominated by multiprocessor systems, which typically have two or more processors, each with a single-core CPU.
- These processors share resources such as the computer bus, clock, memory, and peripheral devices.
- The primary benefit of multiprocessor systems is increased throughput, enabling more work to be done in less time.
- However, the speed-up ratio is less than the number of processors due to overhead from coordination and contention for shared resources, which reduce the efficiency of additional processors.

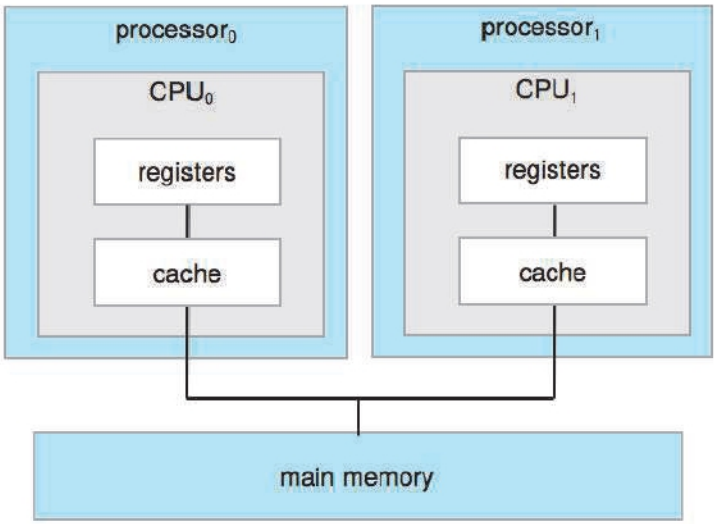


Figure 1.8 Symmetric multiprocessing architecture.

Computer System Architecture

Multi Processor System

- Also called as Shared Memory Processor(SMP) systems have multiple peer CPUs, each capable of performing all tasks, including operating system and user processes. Each CPU has its own registers and private cache while sharing physical memory over a system bus.
- SMP enables simultaneous execution of multiple processes—up to N processes for N CPUs—without significant performance degradation.
- Inefficiencies may arise if workloads are not evenly distributed among CPUs. SMP systems can dynamically share processes and resources like memory across processors, reducing workload imbalances. This requires careful system design to ensure efficient operation

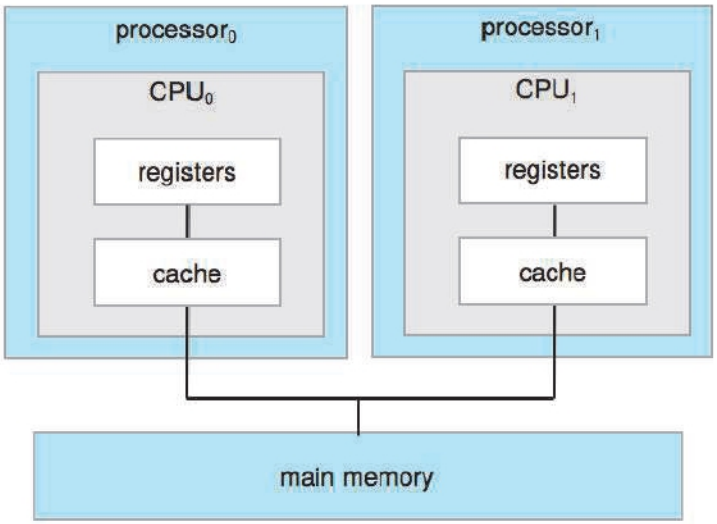


Figure 1.8 Symmetric multiprocessing architecture.

Computer System Architecture

Multi Processor System-UMA Bus-Based

- Although all multiprocessors have the property that every CPU can address all of memory, some multiprocessors have the additional property that every memory word can be read as fast as every other memory word. These machines are called **UMA** (**Uniform Memory Access**) multiprocessors.

UMA Bus-Based SMP(Shared Memory Multiprocessor) Architectures

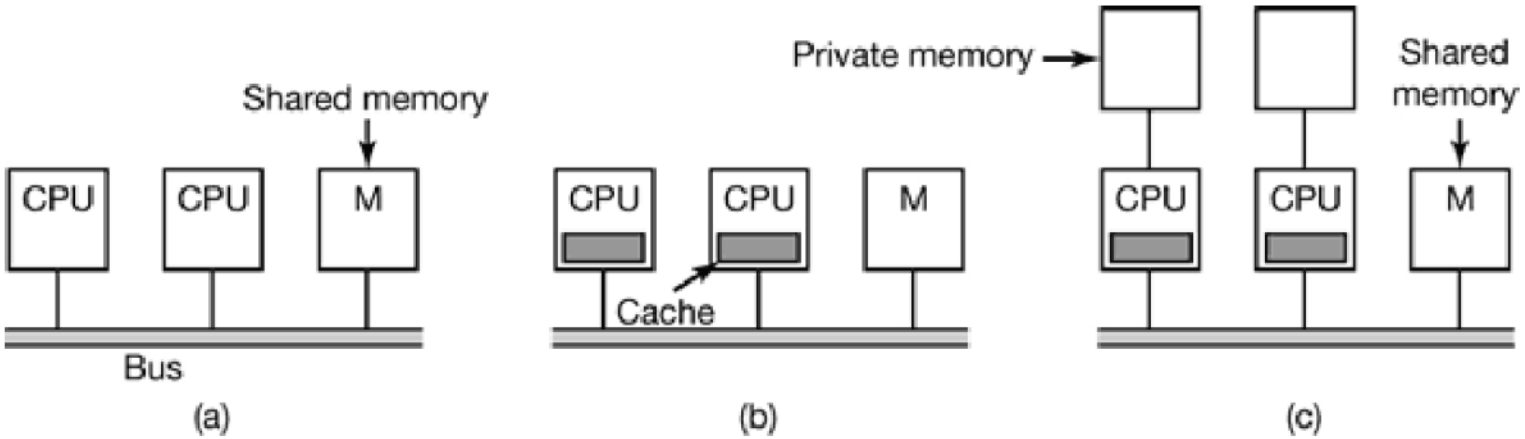


Figure 8-2. Three bus-based multiprocessors. (a) Without caching. (b) With caching. (c) With caching and private memories.

Computer System Architecture

UMA Bus-Based SMP Architectures

- The simplest multiprocessor systems use a single bus for communication between CPUs and memory modules.
- Two or more CPUs and one or more memory modules all use the same bus for communication
- When a CPU wants to read a memory word, it first checks to see if the bus is busy. If the bus is idle, the CPU puts the address of the word it wants on the bus, asserts a few control signals, and waits until the memory puts the desired word on the bus. If the bus is busy when a CPU wants to read or write memory, the CPU just waits until the bus becomes idle.
- However, with a small number of CPUs (e.g., 2-3), contention for the bus is manageable. As the number of CPUs increases (e.g., 32-64), contention becomes severe, leading to idle CPUs and degraded performance.
- Adding a cache to each CPU significantly reduces bus traffic by allowing local cache hits for many memory accesses. This enables the system to support more CPUs without overwhelming the bus.

Computer System Architecture

UMA Bus-Based SMP Architectures

- Caches can be located inside the CPU chip, near the CPU chip, on the processor board, or a combination of these locations. However, an issue of Cache Coherence is observed i.e. When multiple processors modify a shared data that is cached in their local caches, inconsistencies(cached copy and main memory copy are different) can arise if changes are not communicated correctly
- Cache coherency ensures Read Consistency, Write Propagation and Serialization of Writes.
- Each cache block is marked as being either read-only (in which case it can be present in multiple caches at the same time), or as read-write (in which case it may not be present in any other caches).

Computer System Architecture

UMA Bus-Based SMP Architectures

- If a CPU attempts to write a word(in memory) that is in one or more remote caches, the bus hardware detects the write and puts a signal on the bus informing all other caches of the write.
- If other caches have a “clean” copy, that is, an exact copy of what is in memory, they can just discard their copies and let the writer fetch the cache block from memory before modifying it.
- If some other cache has a “dirty” (i.e., modified) copy, it must either write it back to memory before the write can proceed or transfer it directly to the writer over the bus.

.

Computer System Architecture

Multiprocessor System: NUMA Based

- Adding more CPUs to a multiprocessor system increases computing power, but scalability is limited due to contention for the system bus, which can become a performance bottleneck.
- Non-uniform memory access (NUMA) systems address scalability issues by giving each CPU (or group of CPUs) its own local memory, accessed via a fast local bus, while sharing a physical address space through a system interconnect.

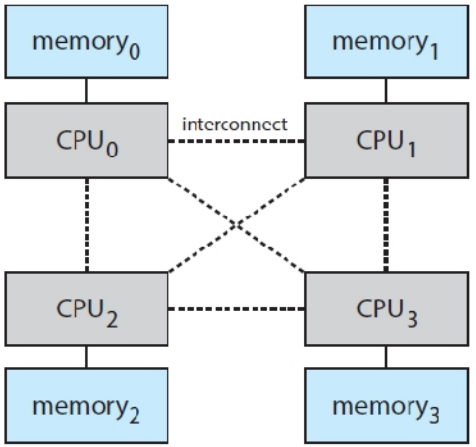


Figure 1.10 NUMA multiprocessing architecture.

Computer System Architecture

NUMA System

- Single-bus UMA multiprocessors are generally limited to no more than a few dozen CPUs and are not that much bigger.
- To get to more than 100 CPUs, something has to give. Usually, what gives is the idea that all memory modules have the same(**Uniform**) access time. This concern leads to the idea of NUMA multiprocessor.
- Like their UMA cousins, they provide a single address space across all the CPUs, but unlike the UMA machines, access to local memory modules is faster than access to remote ones.
- Thus, all UMA programs will run without change on NUMA machines, but the performance will be worse than on a UMA machine at the same clock speed. Example, On a NUMA machine, such programs might allocate data in memory regions distant from the processor accessing them, causing unnecessary remote memory accesses.

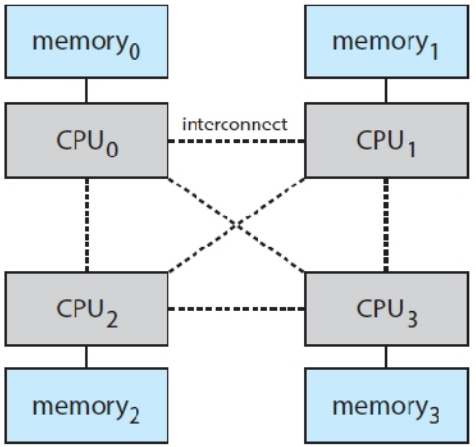


Figure 1.10 NUMA multiprocessing architecture.

Computer System Architecture

NUMA System

- NUMA systems reduce contention and allow faster local memory access, making them more scalable and suitable for large multiprocessor systems.
- Accessing remote memory across the system interconnect introduces latency, which can degrade performance.
- Operating systems mitigate this by optimizing CPU scheduling and memory management.

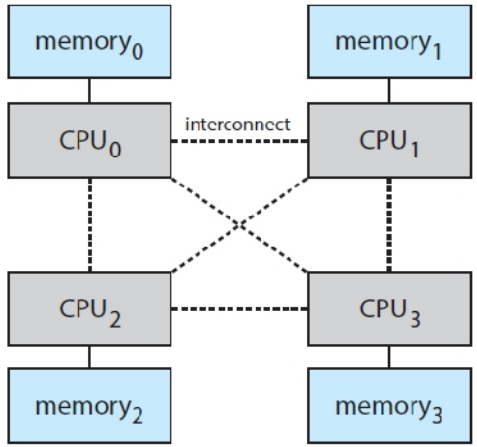


Figure 1.10 NUMA multiprocessing architecture.

Computer System Architecture

NUMA System

- Main three Characteristics:

1. There is a single address space visible to all CPUs.
2. Access to remote memory is via LOAD and STORE instructions.
3. Access to remote memory is slower than access to local memory.

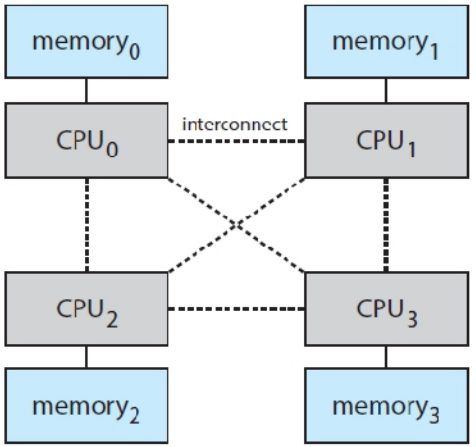


Figure 1.10 NUMA multiprocessing architecture.

Computer System Architecture

Feature	NUMA (Non-Uniform Memory Access)	UMA (Uniform Memory Access)
Memory Access Time	Varies depending on whether a CPU accesses its local memory (fast) or remote memory (slower).	Equal memory access time for all CPUs, as all CPUs share the same memory equally.
Memory Architecture	Each CPU (or group of CPUs) has its own local memory, connected via a system interconnect.	A single, centralized memory is shared among all CPUs.
Performance	Higher scalability due to reduced contention for memory access when using local memory.	Limited scalability due to increased contention for the shared memory as the number of CPUs grows.
Scalability	Scales better for larger systems with many CPUs, often used in high-performance computing.	Less suitable for systems with many CPUs due to bottlenecks in shared memory access.
Latency	Local memory access is fast, but remote memory access can have high latency.	Uniform latency for all memory accesses.
Use Cases	Common in large-scale servers and high-performance computing environments.	Suitable for smaller systems with fewer CPUs or where uniform memory access is critical.
System Design	Requires advanced operating system support for memory management and CPU scheduling.	Simpler system design as all CPUs access the same memory equally.
Cost	More expensive due to the need for additional hardware like local memory for each CPU.	Less expensive to implement, as all CPUs share the same memory.

Computer System Architecture

Multicore System

- Modern multiprocessor systems include multicore architectures, where multiple cores are integrated on a single chip.
- Multicore systems are more efficient than multiple single-core chips due to faster on-chip communication compared to between-chip communication.
- A single chip with multiple cores consumes significantly less power than multiple single-core chips, making it ideal for power-sensitive devices like laptops and mobile devices.
- In a dual-core processor, each core has its own register set and a local (L1) cache. Additionally, the cores share a larger, higher-level (L2) cache on the same processor chip.

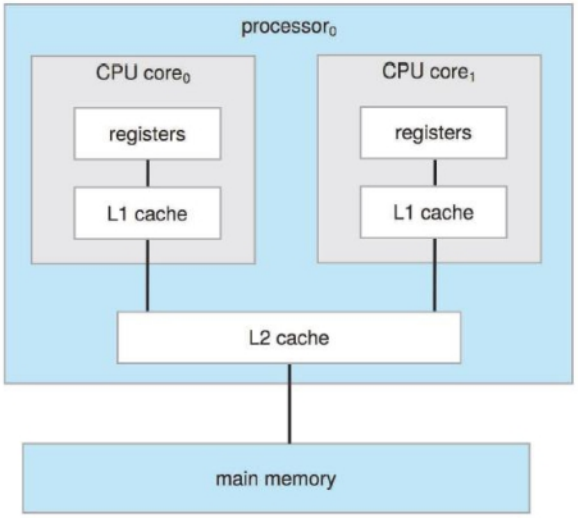


Figure 1.9 A dual-core design with two cores on the same chip.

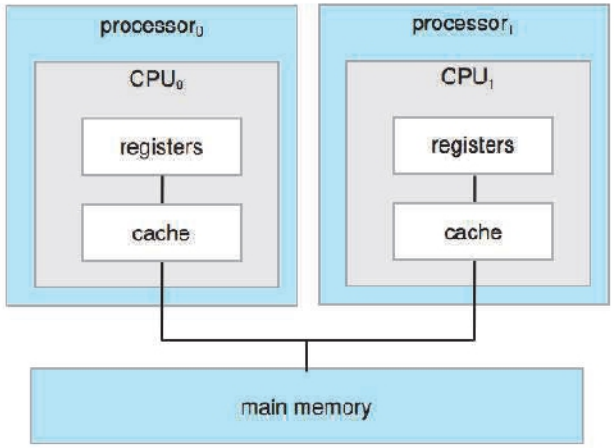


Figure 1.8 Symmetric multiprocessing architecture.

Computer System Architecture

Multicore System

- Most multicore systems combine local, smaller, and faster caches with shared, larger, and slower caches to balance performance and efficiency in data access.
- A multicore processor with N cores appears to the operating system as N separate CPUs, requiring the OS and applications to optimize their design to effectively utilize these cores.
- Operating systems like Windows, macOS, Linux, Android, and iOS are designed to support multicore symmetric multiprocessing (SMP) systems, enabling efficient use of multicore architectures.

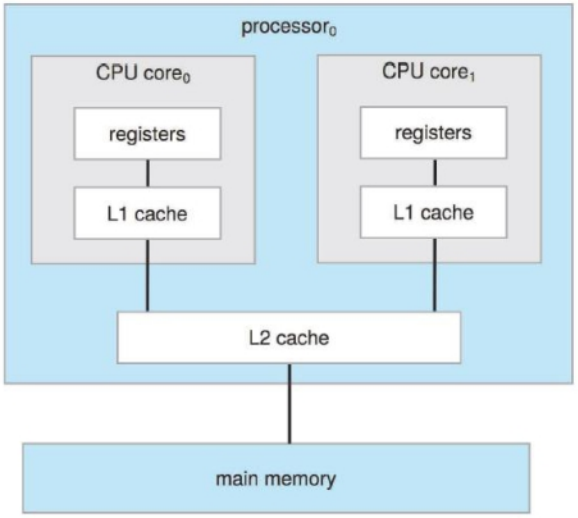


Figure 1.9 A dual-core design with two cores on the same chip.

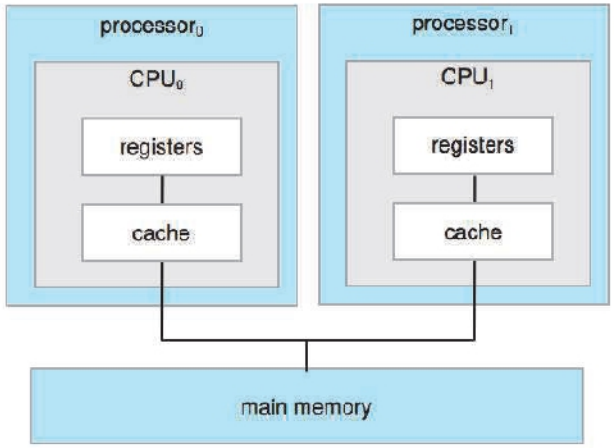


Figure 1.8 Symmetric multiprocessing architecture.

Multiprocessor Scheduling

- On a uniprocessor, scheduling is one dimensional. The only question that must be answered (repeatedly) is: "Which process should be run next?"
- On a multiprocessor, scheduling is two dimensional. The scheduler has to decide which process to run and which CPU to run it on. This extra dimension greatly complicates scheduling on multiprocessors.
- Traditionally, the term multiprocessor referred to systems that provided multiple physical processors, where each processor contained one single-core CPU. However, the definition of multiprocessor has evolved significantly, and on modern computing systems, multiprocessor now applies to the following system architectures:
  - Multicore CPUs
  - Multithreaded cores
  - NUMA systems

Multiprocessor Scheduling

Approaches to Multiprocessor Scheduling

1. Asymmetric Multiprocessing (AMP)

- In this approach, all scheduling decisions, I/O processing, and other system activities are handled by a single processor (master server).
- Other processors execute only user code.
- Advantage: Simple design as only one core accesses system data structures, reducing the need for data sharing.
- Disadvantage: The master server becomes a potential bottleneck, reducing overall system performance.

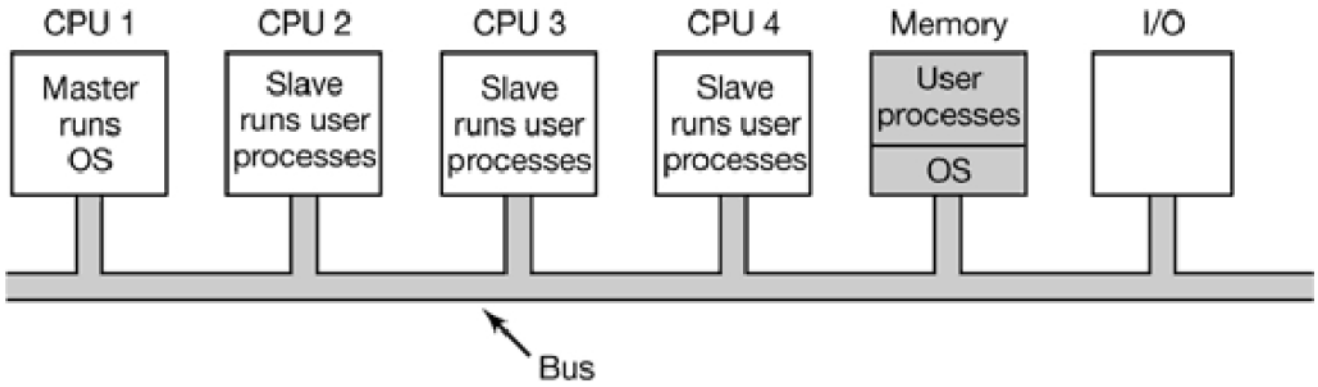


Figure 8-8. A master-slave multiprocessor model.

Multiprocessor Scheduling

2. Symmetric Multiprocessing(SMP)

- The standard scheduling approach for supporting multiprocessors is symmetric multiprocessing (SMP), where each processor is self-scheduling i.e. examining the ready queue and selecting a process/thread to run.
- Note that this provides two possible strategies for organizing the threads eligible to be scheduled.
    1. All threads may be in a common ready queue i.e. Global Ready Queue
    2. Each processor may have its own private queue of threads i.e. Per Processor Ready Queue

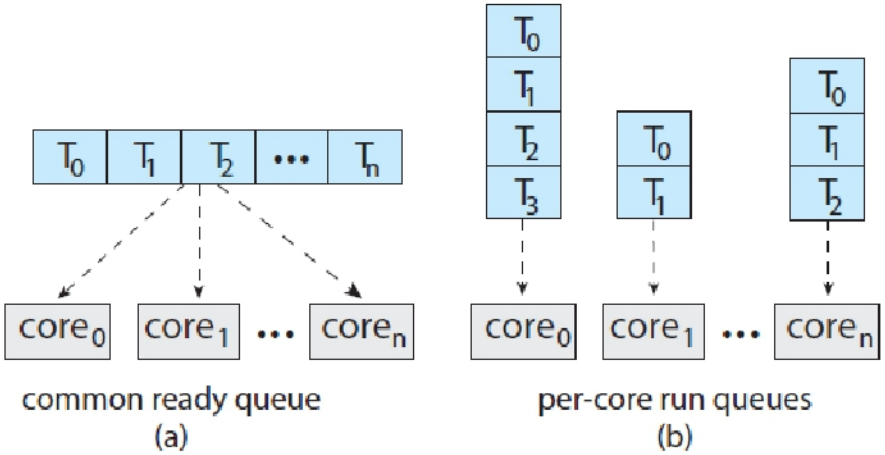


Figure 5.11 Organization of ready queues.

Multiprocessor Scheduling

Symmetric Multiprocessing(SMP)

- In first case, we have a possible race condition on the shared ready queue and therefore, must ensure that two separate processors do not choose to schedule the same thread and that threads are not lost from the queue.
- To avoid race condition, accessing a shared queue requires lock ownership, however this leads to high contention and making it a potential performance bottleneck.
- The second option permits each processor to schedule threads from its private run queue and therefore does not suffer from the possible performance problems associated with a shared run queue.
- Per-processor run queues not only avoid performance issues of shared queues but also enhance cache memory efficiency.
- Cache Affinity: Threads scheduled on the same processor repeatedly can reuse data already in that processor's cache, improving memory access speed and reducing cache misses.
- Scheduling in SMP systems involves designing algorithms(load balancing algorithms) for efficient thread allocation and workload management.
- Thus, it is the most common approach on systems supporting SMP

Multiprocessor Scheduling

Multicore Processor Scheduling

- Traditionally, SMP systems have allowed several processes to run in parallel by providing multiple physical processors.
- However, most contemporary computer hardware now places multiple computing cores on the same physical chip, resulting in a multicore processor.
- Each core maintains its architectural state and thus appears to the operating system to be a separate logical CPU.
- Multicore processors may complicate scheduling issues.
- **Memory stalls** occur when processors wait for data, often due to slower memory speeds or cache misses, leading to performance issues.
- Processors may spend up to 50% of their time waiting for data, causing delays in computation.
- To alleviate memory stalls, modern processors implement **multithreaded cores,** where each core handles multiple hardware threads.

Multiprocessor Scheduling

Multicore Processor Scheduling

- When one thread stalls, the core can switch to another active thread, keeping the processor busy.
- Cache and memory optimization in multicore scheduling plays a vital role in maximizing the performance of multicore systems. It revolves minimizing memory access times, reducing cache misses, and efficiently utilizing multicore caches and memory.

Multiprocessor Scheduling

Multicore Processor Scheduling

- Each hardware thread in a multithreaded core maintains its own state, appearing as a logical CPU to the operating system.
- Intel's Hyper-Threading assigns multiple hardware threads to a single core, effectively increasing the number of logical CPUs. For example: 8 logical CPUs with 4 Core, 2 Thread Per Core shown is aside diagram

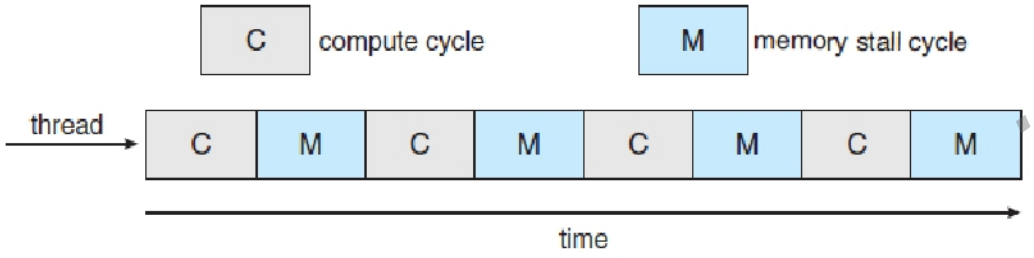


Figure 5.12 Memory stall.

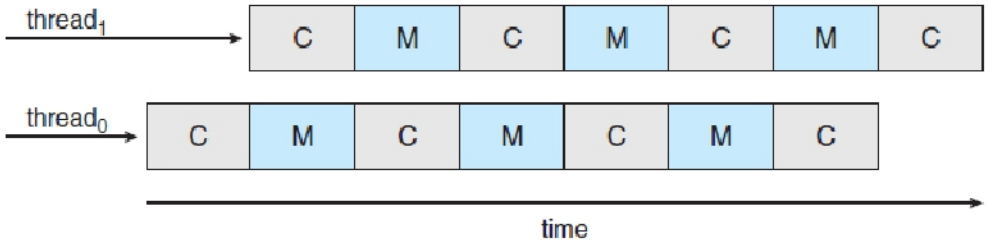


Figure 5.13 Multithreaded multicore system.

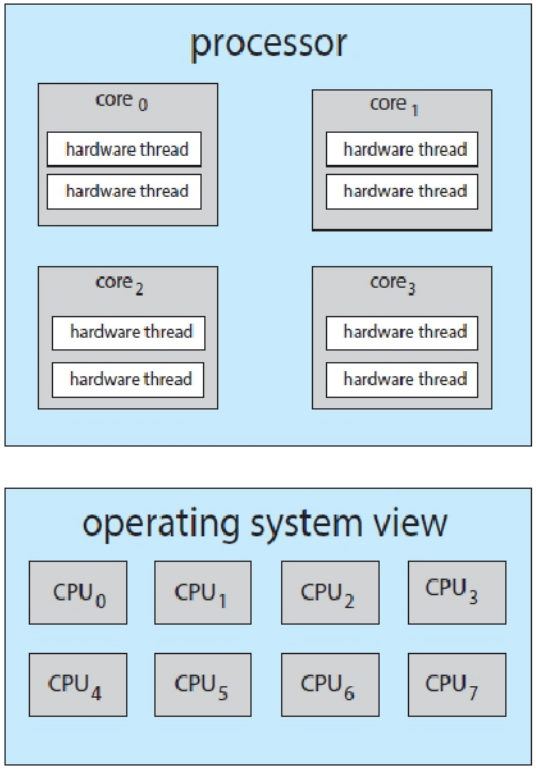


Figure 5.14 Chip multithreading.

Multiprocessor Scheduling

Multicore Processor Scheduling

- The resources of the physical core (such as caches) must be shared among its hardware threads, and therefore a processing core can only execute one hardware thread at a time.
- Consequently, a multithreaded, multicore processor actually requires two different levels of scheduling as show in figure below.

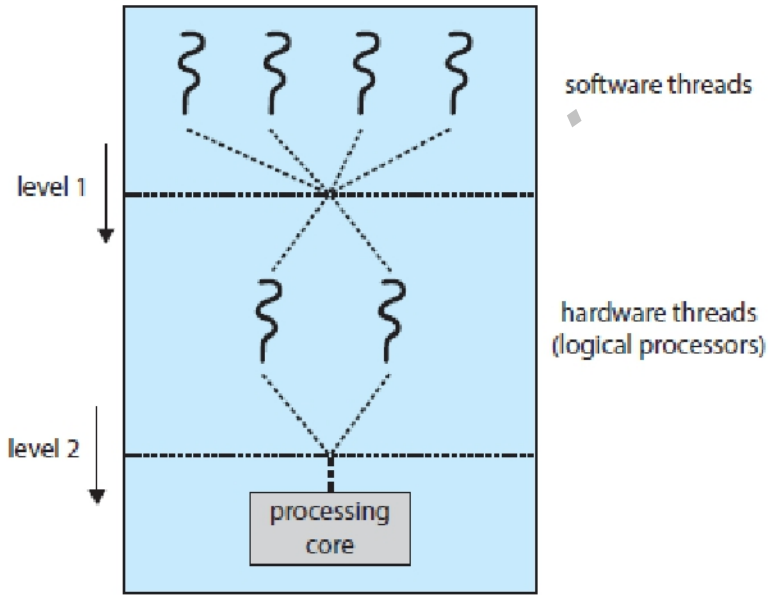


Figure 5.15 Two levels of scheduling.

Multiprocessor Scheduling

Multicore Processor Scheduling

- In first level of scheduling, Operating systems make scheduling decisions to determine which software thread runs on each hardware thread (logical CPU). The operating system can use any scheduling algorithm like FCFS, RR, SJF etc. for making scheduling decisions.
- The second level of scheduling determines which hardware thread to run on each core. One strategy is using a round-robin algorithm, as implemented by the Ultra SPARC T3 processor. Another approach, used by the Intel Itanium, assigns each hardware thread a dynamic urgency value ranging from 0 to 7. When a thread switch occurs, the thread with the higher urgency value is selected for execution on the core.

Multiprocessor Scheduling

Multicore Processor Scheduling

- The two levels of scheduling (operating system scheduling and hardware thread scheduling) **can work together to improve performance**.
- If the operating system knows how processor resources are shared, it can make better decisions.
- For example, if a CPU has two cores, each with two hardware threads, and there are two software threads, the operating system can choose to run them on separate cores.
- If both threads run on the same core, they have to share resources, which can slow them down.
- Knowing this, the operating system can schedule the threads on different cores to avoid resource sharing and improve performance.

Multiprocessor Scheduling

Load Balancing

- In Symmetric Multiprocessing systems, workload balancing is crucial to fully utilize all processors. Without load balancing, some processors may remain idle while others handle high workloads, leading to inefficiency. Load balancing aims to distribute the workload evenly across all processors.
- This is mainly needed in systems where each processor has its own private ready queue of threads. In systems with a common ready queue, load balancing isn't necessary because idle processors can directly fetch a runnable thread from the shared queue .
- There are two main approaches to load balancing: push migration and pull migration.
- Push migration involves a task checking the load on each processor and moving threads from overloaded to idle processors.
- Pull migration occurs when an idle processor pulls tasks from a busy processor.

Multiprocessor Scheduling

Load Balancing

- Push and pull migration can be implemented together in a system for more efficient load balancing.
- The concept of a "balanced load" can vary, either meaning equal numbers of threads in each queue or an even distribution of thread priorities across queues. In some cases, these strategies may not be enough.

Push Migration Example

Imagine a system with 4 processors (P1, P2, P3, P4) and the following initial load distribution.

Processor	Tasks
P1	8
P2	5
P3	3
P4	2

Multiprocessor Scheduling

Push Migration Example

- In this case, Processor P1 is overloaded, and P3 and P4 are under-loaded.
- A task periodically checks the load on each processor. It finds that P1 has 8 tasks, while P4 has only 2 tasks, so P1 is overloaded.
- The load balancing task will move some tasks from P1 to P3 and P4 to balance the load.
- Let's say 3 tasks are moved from P1 to P4, and 2 tasks are moved from P1 to P3.

Processor	Tasks
P1	3
P2	5
P3	5
P4	5

- Now the load is more balanced across the processors, with each processor having 3 to 5 tasks.

Multiprocessor Scheduling

Pull Migration Example

- Let's assume the same setup of 4 processors, but this time with pull migration, where an idle processor (P4) pulls tasks from a busy processor (P1):

Processor	Tasks
P1	10
P2	6
P3	4
P4	0

- P4, which is idle, checks the load on other processors and finds that P1 has 10 tasks.
- P4 decides to pull tasks from P1. Since it's idle, it may pulls 4 tasks from P1.
- New task distribution:

Processor	Tasks
P1	6
P2	6
P3	4
P4	4

- After the pull migration, P1's load is reduced, and P4 has a balanced number of tasks.

Multiprocessor Scheduling

Processor Affinity

- When a thread runs on a processor, the data it accesses populate the processor's cache, making subsequent memory accesses faster (a "warm cache").
- If the thread migrates to another processor, the first processor's cache must be invalidated, and the second processor's cache must be repopulated.
- The process of invalidating and repopulating caches is costly in terms of performance.
- To avoid this cost, many operating systems try to keep a thread on the same processor, maintaining the warm cache.
- This approach is called **processor affinity**, where a process has an affinity for the processor it is currently running on.

Multiprocessor Scheduling

Processor Affinity

- The two strategies in SMP for organizing the queue(Common Ready Queue and Per Core Run Queue) of threads available for scheduling have implications for processor affinity.
- If we adopt the approach of a common ready queue, a thread may be selected for execution by any processor. Thus, if a thread is scheduled on a new processor, that processor's cache must be repopulated.
- With private, per-processor ready queues, a thread is always scheduled on the same processor and can therefore benefit from the contents of a warm cache.
- Essentially, per-processor ready queues provide processor affinity for free!
- Processor affinity takes several forms: Soft Affinity and Hard Affinity
- In Soft Affinity, the operating system attempts to keep a process on the same processor, but migration between processors is possible during load balancing. So here the operating system do not guarantee that a process is always kept on same processor.

Multiprocessor Scheduling

Processor Affinity

- In Hard affinity, Some systems allow processes to specify a subset of processors they can run on, offering more control over process placement.
- So here, a process can be restricted to running only on certain processors rather than being allowed to run on any processor.
- For example, in a multi-core system with 8 processors, a process may be configured to run only on processors 2, 4, and 6, forming a subset of the total available processors.
- This ensures the process does not migrate to other processors outside the specified subset.

Multiprocessor Scheduling

NUMA Aware Processor Scheduling

- The main-memory architecture of a system can affect processor affinity issues as well.
- The below figure illustrates an architecture featuring non-uniform memory access (NUMA) where there are two physical processor chips each with their own CPU and local memory.

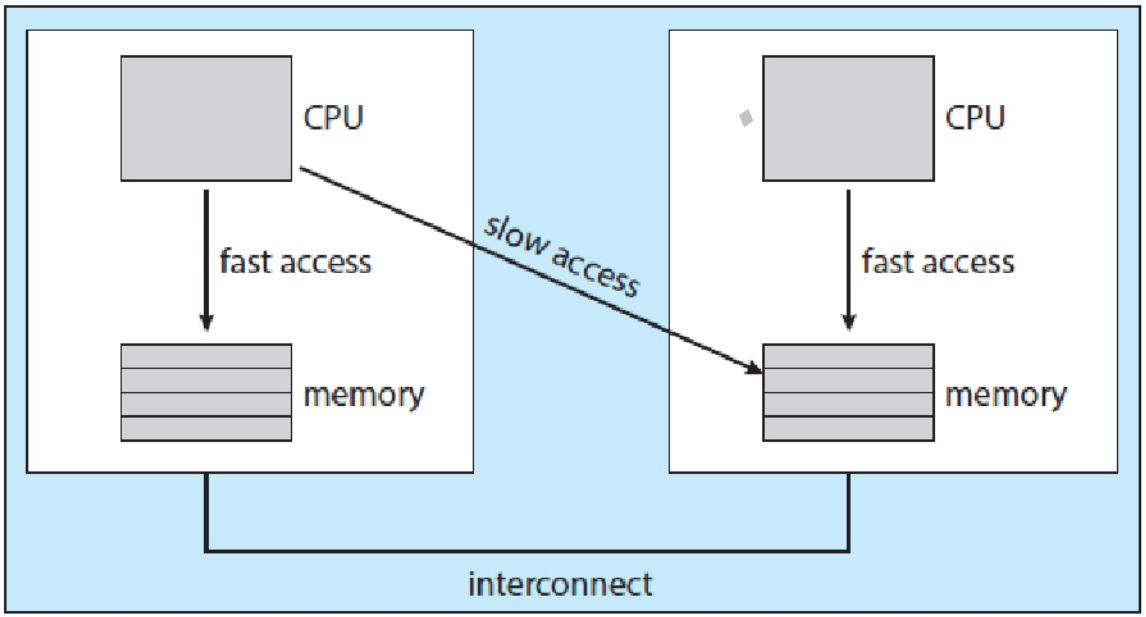


Figure 5.16 NUMA and CPU scheduling.

Multiprocessor Scheduling

NUMA Aware Processor Scheduling

- Although a system interconnect allows all CPUs in a NUMA system to share one physical address space, a CPU has faster access to its local memory than to memory local to another CPU.
- If the operating system's CPU scheduler and memory-placement algorithms are NUMA-aware and work together, they can optimize thread scheduling and memory allocations i.e. a thread that has been scheduled onto a particular CPU can be allocated memory closest to where the CPU resides, thus providing the thread the fastest possible memory access.
- Interestingly, load balancing often counteracts the benefits of processor affinity. Load balancing can reduce the benefits of processor affinity by moving threads between processors, which disrupts cache utilization.
- Migrating a thread between processors may incur a penalty on NUMA systems, where a thread may be moved to a processor that requires longer memory access times.
- In other words, there is a natural tension between load balancing and minimizing memory access times. Thus, scheduling algorithms for modern multicore NUMA systems have become quite complex.

Thread Libraries and Synchronization

- Thread libraries provide essential tools/functions for creating and managing threads in a program. These libraries offer interfaces and functions to create, manage, and control threads. They abstract the complexities of thread management, providing developers with easy-to-use APIs to handle concurrency.
- Threads, as units of execution, allow programs to perform multiple tasks concurrently(multithreading), which is vital for improving performance, particularly in multi-core systems.
- However, managing multiple threads requires synchronization techniques to avoid issues like race conditions, deadlocks, and resource conflict.
- The following example demonstrates demonstrate race condition issues in multi threaded environment.

Thread Libraries and Synchronization

- When two or more concurrently running threads access a shared data item and the final result depends on the order of execution, we have a race condition.
- Suppose, we have two threads A and B as shown below. Thread A increases the shared variable Count by 1 and thread B decreases Count by 1.
- Since Count is shared by two threads, we know it should be protected by a mutex lock or a binary semaphore.

```
Thread A                Thread B
.....                   .....
Count++;                Count--;
.....                   .....
```

Thread Libraries and Synchronization

- When two or more concurrently running threads access a shared data item and the final result depends on the order of execution, we have a race condition.
- Suppose, we have two threads A and B as shown below. Thread A increases the shared variable Count by 1 and thread B decreases Count by 1.
- Since Count is shared by two threads, we know it should be protected by a mutex lock or a binary semaphore.

```
Thread A          Thread B
.....             .....
Count++;          Count--;
.....             .....
```

- If Count is not protected by mutual exclusion(mutex or lock), we might get difference results. Statements Count++ and Count-- may be translated to machine instructions as shown below:

Thread Libraries and Synchronization

Thread A	Thread B
.....	.....
LOAD Count	LOAD Count
ADD #1	SUB #1
STORE Count	STORE Count
.....	.....

- If both statements are not protected, the execution of the instructions may be interleaved due to context switching. The following table shows the execution details.

Thread A		Thread B		
Instruction	Register	Instruction	Register	Count
LOAD Count	10			10
		LOAD Count	10	10
		SUB #1	9	10
		STORE Count	9	9
ADD #1	11			10
STORE Count	11			11

- First Thread A is running. After thread A executes its LOAD instruction, a context switch switches thread A out and switches thread B in. Thus, thread B executes its three instructions, changing the value of Count to 9.Then, the execution is switched back to thread A, which continues with the remaining two instructions. Consequently, the value of Count is changed to 11.

Thread Libraries and Synchronization

- The following shows the execution flow of executing thread B followed by thread A, and the result is 9.

Thread A		Thread B		
Instruction	Register	Instruction	Register	Count
		LOAD Count	10	10
LOAD Count	10			10
ADD #1	11			10
STORE Count	11			11
		SUB #1	9	11
		STORE Count	9	9

- This example shows that without mutual exclusion, the access to a shared data item may generate different results if the execution order is altered. This is, of course, a race condition.
- This race condition is due to no mutual exclusion i.e. due to missing synchronization

Thread Libraries and Synchronization

Thread Synchronization Protocols

- Thread synchronization protocols are mechanisms and techniques used to coordinate the execution of threads in a multithreaded environment to ensure correct, predictable, and efficient access to shared resources or data.
- These protocols prevent race conditions, deadlocks, and data corruption by controlling how threads interact with shared resources.
- Some of the protocols includes
    1. Conditional Variable based synchronization
    2. Semaphore based Synchronization
    3. Adaptive lock
    4. Lock Free Data Structure

Thread Libraries and Synchronization

Conditional Variable Based Synchronization

- Before Going Through Conditional Variable lets have basic idea **on spin lock.**
- A spin lock is a synchronization mechanism used in multithreaded programming to prevent multiple threads from accessing a shared resource simultaneously.
- Unlike other locks (e.g., mutexes), a spin lock does not put a thread to sleep while waiting for the lock to become available.
- Instead, the thread "spins" in a loop, repeatedly checking if the lock is available.

How it work?

- A spin lock is usually implemented as a single atomic variable, often an integer or Boolean. Example: 0 for "unlocked" and 1 for "locked."

Thread Libraries and Synchronization

Spin Lock

- A thread attempts to acquire the lock using an **atomic test-and-set** operation.
- If the lock is already held by another thread, the thread keeps spinning in a loop, checking until the lock becomes available.
- Once the lock is acquired, the thread enters the critical section(Section where shared resources are being updated like database, variable etc.) and safely accesses or modifies the shared resource.
- After completing the critical section, the thread releases the lock by setting it back to "unlocked" using an atomic operation.

Thread Libraries and Synchronization

Spin Lock

Advantage:

1. Simple to implement compared to other synchronization primitives
2. They **do not involve context switches** or putting threads to sleep, making them faster in scenarios where the lock is held for a very short time.

Disadvantage:

1. Threads **waste CPU cycles** while spinning, which can lead to poor performance in high-contention scenarios.
2. If the lock is held for a long time, it is more efficient to use a sleep-based locking mechanism like a mutex.

Thread Libraries and Synchronization

Using POSIX Thread Library in C, Create a single thread that displays all the even numbers from 1 to 100 and print at an interval of 2 seconds.

```c
#include <pthread.h>
#include <stdio.h>
#include<unistd.h>//for sleep function
int counter=0;
void* Even(void* arg) {  //Create Even thread
    for(int i=2;i<=100;i=i+2)
    {
        printf("%d\t",i);
        sleep(2);
    }
    return NULL;
}
int main() {//Main Thread
    printf("parent Begins Here \n");
    pthread_t even;  // Declare the thread handle
    pthread_create(&even, NULL, Even, NULL);// Create the child thread
    // Wait for the child thread to finish
    pthread_join(even,NULL);  // This will block the parent until the child thread finishes
    printf("\nParent Ends Here \n");  // This will be printed after the child finishes
    return 0;
}
```

This is single threaded so we will get expected output

```
parent Begins Here
2       4       6       8       10      12      14      16      18      20      22      24      26      28      30
        32      34      36      38      40      42      44      46      48      50      52      54      56      58
        60      62      64      66      68      70      72      74      76      78      80      82      84      86
        88      90      92      94      96      98      100
Parent Ends Here
```

Thread Libraries and Synchronization

In the above program

- pthread.h: It is a header file in C that provides an interface for the POSIX (Portable Operating System Interface) threads library, commonly known as pthread. It defines functions, constants, and data types that enable the creation, synchronization, and management of threads in a C program.
- pthread_create: Creates a new thread in the current process. It returns 0 on success, and an error number if it fails.
- pthread_join: Waits for the specified thread to terminate. This function makes the calling thread (the main thread in this case) wait for the specified thread (child thread Even) to finish its execution. The second argument (NULL in this case) can be used to retrieve the return value of the joined thread if needed, but it's not used here.
- Sleep: Suspends the execution of the calling thread for a specified number of seconds. The sleep function causes the calling thread (the Even thread in this case) to pause execution for the specified time in seconds.

Thread Libraries and Synchronization

Using POSIX Thread Library in C, Create a multiple thread that displays all the even and numbers from 1 to 10 and print at an interval of 1 seconds.

```c
#include <pthread.h>
#include <stdio.h>
#include<unistd.h>//for sleep function
void* Even(void* arg) {  //Create Even thread
    for(int i=2;i<=100;i=i+2)
    {
        printf("%d\t",i);
        sleep(1);
    }
    return NULL;
}
void* Odd(void* arg) {  //Create Odd thread
    for(int i=1;i<=100;i=i+2)
    {
        printf("%d\t",i);
        sleep(1);
    }
    return NULL;
}
int main() {//Main Thread
    printf("parent Begins Here \n");
    pthread_t even,odd;  // Declare the thread handle
    pthread_create(&even, NULL, Even, NULL);// Create the child thread
    pthread_create(&odd, NULL, Odd, NULL);// Create the child thread
    pthread_join(even,NULL);  // This will block the parent until the child thread finishes
    pthread_join(odd,NULL);  // This will block the parent until the child thread finishes
    printf("Parent Ends Here \n");  // This will be printed after the child finishes
    return 0;
}
```

Since, thread are independent, so even and odd numbers are printed as expected

```
parent Begins Here
2       4       6       8       10      12      1       3       5       14      16      18      20      22      24
        7       9       11      13      26      28      30      32      34      36      38      40      42      44
        46      48      50      52      54      56      58      60      62      64      66      68      70      72
        74      76      78      80      82      84      86      88      90      92      94      96      98      100
        15      17      19      21      23      25      27      29      31      33      35      37      39      41
        43      45      47      49      51      53      55      57      59      61      63      65      67      69
        71      73      75      77      79      81      83      85      87      89      91      93      95      97
        99      Parent Ends Here
```

Thread Libraries and Synchronization

Now let's look the following example:

```c
#include <pthread.h>
#include <stdio.h>
#include<unistd.h>//for sleep function
int counter=0;
void* Addition(void* arg) {  //Create Even threads
    //sleep(5);
    int val=counter;
    val=val * 2;
    counter=counter + val;
    return NULL;
}
void* Subtraction(void* arg) {  //Create Odd thread
    //sleep(5);
    int val=counter;
    val=val -2;
    counter= counter - val;
    return NULL;
}
int main() {//Main Thread
    printf("parent Begins Here \n");
    pthread_t add,sub;  // Declare the thread handle
    pthread_create(&add, NULL, Addition, NULL);// Create the child thread
    pthread_create(&sub, NULL, Subtraction, NULL);// Create the child thread

    // Wait for the child thread to finish
    pthread_join(add,NULL);  // This will block the parent until the child thread finishes
    pthread_join(sub,NULL);  // This will block the parent until the child thread finishes
    printf("Count Value=%d",counter);
    printf("Parent Ends Here \n");  // This will be printed after the child finishes
    return 0;
}
```

In this example, counter is a shared between two thread Addition and Subtraction. If this program is executed then in different run , we get different forms of output i.e. subjected to race condition. This arises the need of thread synchronization.

Output for two different runs

```
parent Begins Here
Count Value=2Parent Ends Here
```

```
parent Begins Here
Count Value=6Parent Ends Here
```

Thread Libraries and Synchronization

Thread Synchronization using Spin Lock

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h> // For sleep function
int counter = 0;
int lock = 0;  // Lock variable to simulate the spin lock
// Create the Addition thread
void* Addition(void* arg) {
    while (lock) {  // Spin lock: Wait until lock is 0 (unlocked)
        // Keep spinning (busy-wait) until lock is released
    }
    lock = 1;  // Acquire the lock
    //sleep(5);  // Simulate work by sleeping
    int val = counter;
    val = val * 2;
    counter = counter + val;
    lock = 0;  // Release the lock
    return NULL;
}

// Create the Subtraction thread
void* Subtraction(void* arg) {
    while (lock) {  // Spin lock: Wait until lock is 0 (unlocked)
        // Keep spinning (busy-wait) until lock is released
    }
    lock = 1;  // Acquire the lock
    //sleep(5);  // Simulate work by sleeping
    int val = counter;
    val = val - 2;
    counter = counter - val;
    lock = 0;  // Release the lock
    return NULL;
}
```

```c
int main() {
    printf("Parent Begins Here \n");
    pthread_t add, sub;
    pthread_create(&add, NULL, Addition, NULL);
    pthread_create(&sub, NULL, Subtraction, NULL);
    pthread_join(add, NULL);
    pthread_join(sub, NULL);
    printf("Count Value = %d\n", counter);
    printf("Parent Ends Here \n");
    return 0;
}
```

Thread Libraries and Synchronization

Conditional Variable based Synchronization

- Conditional variables are a powerful synchronization tool that allow threads to wait for some condition to be met.
- They are often used in situations where a thread needs to wait for a specific condition before continuing execution.
- In C, you can use the **pthread_cond_t type from pthread.h** to implement conditional variables.
- A thread **can wait for a condition to become true by calling pthread_cond_wait().** This makes the thread enter **a blocked (sleeping)** state until the condition is signaled Condition variables maintain an explicit **queue** where threads can wait when a certain condition is not met. When a thread waits, it also releases the mutex (lock) associated with the condition variable, allowing other threads to access the shared resource.
- **A condition variable is always used with a mutex(mutual exclusive Lock) to protect shared resources.** The mutex ensures mutual exclusion and prevents race conditions when checking or modifying the shared state.

Thread Libraries and Synchronization

Conditional Variable based Synchronization

- Another thread can **signal the condition variable when it changes the state, waking up one or more threads that were waiting.** When a thread changes the state or the condition becomes true, it can signal one or more waiting threads to wake up. **This is done using pthread_cond_signal()**
- Once signaled, the waiting threads are allowed to continue execution.

Advantage:

1. They allow threads to **wait without consuming CPU resources**, improving system efficiency.
2. They provide a mechanism for threads to synchronize based on specific conditions, enabling complex thread interactions.

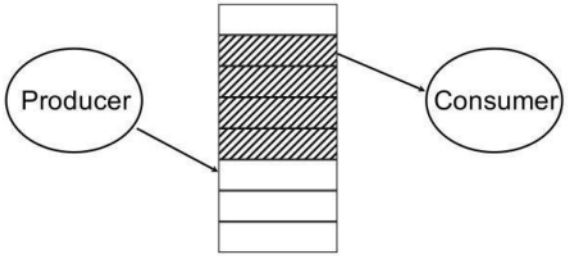
Disadvantage:

1. The use of condition variables introduces some **overhead due to context switching** and thread management, especially in systems with a high number of threads or frequent signaling.
2. If a thread waits on a condition variable indefinitely without getting the required signal, it could lead to a **deadlock situation.**

Thread Libraries and Synchronization

Conditional Variable based Synchronization: Producer Consumer Problem

- In this problem, two processes share a common, fixed-size buffer.
- One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.



- Now the problem arises when
  - The producer works faster than consumer, then the producer wants to put a new data in buffer but all the buffer is full.
  - **Solution:** Producer goes to sleep and to be awakened when the consumer has removed data.
  - The consumer faster than producer, then the consumer wants to remove data the buffer but buffer is already empty.
  - **Solution:** Consumer goes to sleep until the producer puts some data in buffer and wakes consumer up.
- We can solve this problem by using conditional Variable based synchronization as given below.

Thread Libraries and Synchronization

Thread Synchronization using Conditional Variable(Produce Consumer Problem)

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>  // For sleep function
#define BUFFER_SIZE 5  // Size of the buffer
int buffer[BUFFER_SIZE];  // Shared buffer array, fixed size
int count = 0;  // Variable to track the number of items in the buffer
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  // Mutex for shared resource
pthread_cond_t empty = PTHREAD_COND_INITIALIZER;  // Condition variable to signal producer (buffer is not full)
pthread_cond_t full = PTHREAD_COND_INITIALIZER;   // Condition variable to signal consumer (buffer is not empty)

// Producer thread function
void* producer(void* arg) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&lock);  // Acquire lock
        // Wait if buffer is full
        if (count == BUFFER_SIZE) {
            pthread_cond_wait(&empty, &lock);
        }
        // Produce an item (add to buffer)
        buffer[count] = i + 1;
        printf("Producer produced item %d\n", buffer[count]);
        count++;  // Increase the item count

        // Signal the consumer that buffer is not empty
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&lock);  // Release lock
        sleep(1);  // Simulate work by sleeping
    }
    return NULL;
}
```

Thread Libraries and Synchronization

Thread Synchronization using Conditional Variable(Produce Consumer Problem)

```
// Consumer thread function
void* consumer(void* arg) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&lock);  // Acquire lock
        // Wait if buffer is empty
        if (count == 0) {
            pthread_cond_wait(&full, &lock);
        }
        // Consume an item (remove from buffer)
        printf("Consumer consumed item %d\n", buffer[count - 1]);
        count--;  // Decrease the item count

        // Signal the producer that buffer is not full
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&lock);  // Release lock
        sleep(1);  // Simulate work by sleeping
    }
    return NULL;
}
```

Thread Libraries and Synchronization

Thread Synchronization using Conditional Variable(Produce Consumer Problem)

```
int main() {
    pthread_t prod_thread, cons_thread;

    // Create the producer and consumer threads
    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // Wait for threads to finish
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    printf("Main ends\n");
    return 0;
}
```

OUTPUT:

```
Producer produced item 1
Consumer consumed item 1
Producer produced item 2
Consumer consumed item 2
Producer produced item 3
Consumer consumed item 3
Producer produced item 4
Consumer consumed item 4
Producer produced item 5
Consumer consumed item 5
Main ends
```

Thread Libraries and Synchronization

Semaphore Based Thread Synchronization

- A semaphore 'S' is a synchronization tool, which is an integer variable that constitute a mechanism to resolve resource conflicts when several processes access a common resource in concurrent processing environment.
- So, this allow mutual exclusive access to critical section or shared resources.
- A semaphore is an integer variable that can have value 0 to indicate no wake up were saved, or some positive value if one or more wakeups are pending. i.e. all resources are currently in use, and any thread/process trying to access the resource must wait.
- A semaphore variable after being initialized and can be accessed only through two atomic operations. Wait or P for wait to test and Signal or V for signal to release.
- If S is the semaphore variable, then the operation performed by semaphore by using P and V are

Thread Libraries and Synchronization

Semaphore Based Thread Synchronization

P or Wait Operation	V or Signal Operation

P(S)
 	
S--
if(S<0)
{
//place this process in waiting queue
// i.e. block this process: sleep()
}
	V(S)
	
	S++
	If(S<=0)
	{
	//Select process from the queue and put this
	// in ready queue i.e. wake up the
	//process:wakeup()
	
	}

- The down operation on the semaphore checks to see if the value is greater than 0, if so, it decrements the value.
- But if the value is 0, the process is put to sleep without completing down for the moment. The up operation increments the value of semaphore addressed. If one or more processes were sleeping on that semaphore, unable to complete earlier down operation, one of them is chosen by the system (e.g.: FIFO, Random) and is allowed to complete its down.

Thread Libraries and Synchronization

Semaphore Based Thread Synchronization

```
struct semaphore {
        int count;
        queueType queue;
};
void semWait(semaphore s)
{
        s.count--;
        if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
        }
}
void semSignal(semaphore s)
{
        s.count++;
        if (s.count<= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
        }
}
```

Fig: Semaphore Structure and Operations

Thread Libraries and Synchronization

Semaphore Based Thread Synchronization

- Characteristics of Semaphore
  1. Semaphore are like integer variable.
  2. Only P and V operations is possible on semaphore.
  3. The operation P and V are atomic

- Types

1. Binary Semaphore:
   - A semaphore that are initialized to one and used by two or more processes to ensure that only one of them enter its critical region at same time are called binary semaphore. They are used to acquire locks.
   - Binary semaphores can take only 2 values (0/1).
   - When a resource is available, the process in charge set the semaphore to 1 else 0.

Thread Libraries and Synchronization

Semaphore Based Thread Synchronization

2. Counting Semaphore:

- Counting Semaphore may have value to be greater than one.
- Typically, it is used to allocate resources from a pool of identical resources.

Thread Libraries and Synchronization

Solving Producer and Consumer Problem using semaphore

- The solution to this problem is obtained by using three semaphore variable. We assumed a N buffer free

1. Binary Semaphore **'Mutex',** provides mutual exclusion so that both producer and consumer do not access the buffer at same time, and is initialized to 1. So to achieve mutual exclusion, the semaphore variable is first initialized to 1 and P () is called before the critical section and V () is called after the critical section.

Semaphore S =1

Wait(S)

<Critical Section>

Signal(S)

1. Counting Semaphore 'Empty' for counting the number of slots that are empty.
2. Counting Semaphore 'full' for counting the number of slots that are full.

Thread Libraries and Synchronization

Solving Producer and Consumer Problem using semaphore

```
#define N 100                       /* number of slots in the buffer */
int semaphore;                      /* semaphores are a special kind of int */
semaphore mutex = 1;                /* controls access to critical region */
semaphore empty = N;                /* counts empty buffer slots */
semaphore full = 0;                 /* counts full buffer slots */

void producer(void)
{
    int item;
    while (TRUE){                   /* TRUE is the constant 1 */
        item = produce_item();      /* generate something to put in buffer */
        down(&empty);               /* decrement empty count */
        down(&mutex);               /* enter critical region */
        insert_item(item);          /* put new item in buffer */
        up(&mutex);                 /* leave critical region */
        up(&full);                  /* increment count of full slots */
    }
}
```

```
void consumer(void)
{
    int item;
    while (TRUE){                   /* infinite loop */
        down(&full);                /* decrement full count */
        down(&mutex);               /* enter critical region */
        item = remove_item();       /* take item from buffer */
        up(&mutex);                 /* leave critical region */
        up(&empty);                 /* increment count of empty slots */
        consume_item(item);         /* do something with the item */
    }
}
```

Thread Libraries and Synchronization

Solving Producer and Consumer Problem using semaphore

- Conditional variable-based solution is considered complex because it requires explicit condition checking and proper use of mutex and condition variables. Compared to this, Semaphore is simple to implement as semaphores directly manage counters for resource availability.

Task:

1. *Implement Producer Consumer problem using any thread(C, Java etc.) libraries.*
2. *Conditional Variable Based Synchronization. Vs Semaphore Based Thread Synchronization*

Thread Libraries and Synchronization

Adaptive Synchronization

- Adaptive synchronization techniques are modern approaches to improve the performance and scalability of systems by dynamically adjusting synchronization mechanisms based on the current system state or workload.
- Adaptive synchronization techniques leverage insights from the current system state, such as contention levels or workload patterns, to optimize synchronization strategies dynamically.
- By tailoring their behavior, these techniques aim to minimize overhead, reduce contention, and improve resource utilization in diverse operational scenarios.
- These methods are particularly effective in systems with highly variable workloads, such as databases, multi-threaded applications, or real-time systems.

Thread Libraries and Synchronization

Adaptive Lock

- Adaptive locks adjust their behavior dynamically based on contention (the number of threads trying to acquire the lock). They aim to balance between performance in low contention and fairness in high contention scenarios.
- Adaptive locks are particularly useful in environments where thread contention varies significantly over time. In scenarios with low contention, these locks may behave like spinlocks to reduce the overhead of context switching.
- Conversely, in high-contention scenarios, they can switch to blocking mechanisms to ensure fairness and prevent threads from wasting CPU cycles.
- This dynamic adjustment makes adaptive locks more efficient than static locking mechanisms, as they optimize resource usage based on the system's current needs.

Thread Libraries and Synchronization

Adaptive Lock

- **Working Principle**
  - When contention is low, adaptive locks may behave as spinlocks (busy-waiting) for quick lock acquisition.
  - When contention is high, they may switch to blocking locks (thread suspension), reducing CPU usage.
- **Advantage**
  - Minimizes the overhead of unnecessary blocking in low-contention scenarios.
  - Energy Efficiency: Reduces busy-waiting in high contention, saving CPU cycles.
- **Disadvantage**
  - Adapting to contention may introduce complexity and slight overhead.
  - Still uses locks, which can cause contention in extreme scenarios.

Thread Libraries and Synchronization

Adaptive Lock

```
struct AdaptiveLock {
    // Internal state variables
    SpinLock spinLock;        // Lightweight spinning mechanism
    Mutex blockLock;          // Blocking mechanism
    ContentionCounter counter; // Tracks contention level
    // Lock method
    void lock() {
        if (isLowContention()) {
            spinLock.acquire(); // Use spinlock if contention is low
        } else {
            blockLock.acquire(); // Switch to blocking mechanism if contention is high
        }
        updateContentionState(); // Adjust contention metrics
    }
    // Unlock method
    void unlock() {
        if (wasUsingSpinLock()) {
            spinLock.release();
        } else {
            blockLock.release();
        }
    }
    // Check if contention is low
    bool isLowContention() {
        // Logic to decide based on contention levels
        return counter.get() < THRESHOLD;
    }

    // Update contention state
    void updateContentionState() {
        // Adjust contention metrics dynamically
        counter.incrementOrReset();
    }
};
```

Fig: Adaptive Lock and Operation

Thread Libraries and Synchronization

Lock Free Data Structure

- Lock-free data structures avoid traditional locks for synchronization, instead relying on some atomic operations provided by modern hardware to ensure consistency.
- This mechanism doesn't require mutual exclusion locks to ensure safe access and modification of shared data by multiple threads.

Key Concept

- Lock-Free: Guarantees that at least one thread will make progress in a finite number of steps, even if other threads are delayed or fail.

Thread Libraries and Synchronization

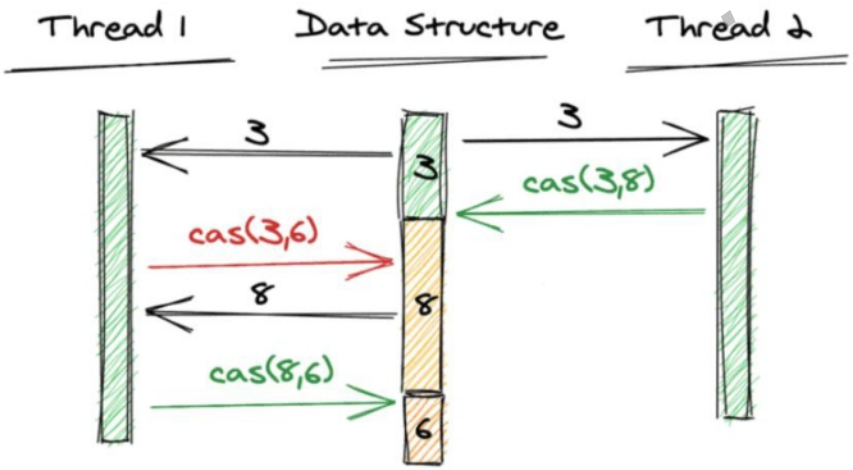
Lock Free Data Structure: Working Principle

- Atomic instructions like Compare-And-Swap (CAS) is used to modify shared data safely.

Example:

CAS(x, expected, newValue):
- Atomically compares x with expected.
- If they are equal, sets x = newValue and returns true.
- Otherwise, returns false, allowing the thread to retry.

•CAS is an atomic operation, which means that fetch and update together are one single operation:



Here, both threads fetch the value 3 from the main memory. Thread 2 succeeds (green) and updates the variable to 8. As the first CAS by thread 1 expects the value to be still 3, the CAS fails (red). Therefore, Thread 1 fetches the value again, and the second CAS succeeds.

Thread Libraries and Synchronization

Lock Free Data Structure: Working Principle

- Atomic operations ensure that updates to shared variables happen atomically, meaning other threads will see either the old value or the fully updated value, but never an intermediate state.
- Threads work **optimistically**, assuming no conflicts with other threads. However, if a conflict is detected (e.g., another thread modified the data), the operation is retried. The failing thread reads the updated state, recomputes its changes, and retries the atomic update.
- Instead of locking an entire data structure, lock-free algorithms ensure synchronization at a smaller scale, typically for individual nodes or segments. This reduces contention and increases parallelism

Thread Libraries and Synchronization

Example: **Lock-free Queue Implemented using linked list**

Enqueue Operation:

- A thread reads the current tail of the queue.
- Prepares a new node to be added.
- Uses CAS to update the tail pointer to point to the new node.
- On CAS failure (tail was updated by another thread), retries with the latest tail value.

Dequeue Operation:

- A thread reads the current head of the queue.
- Verifies that the queue is not empty.
- Uses CAS to update the head pointer to the next node.
- On CAS failure (head was updated by another thread), retries with the latest head value.

Thread Libraries and Synchronization

Lock Free Data Structure: Working Principle

Advantage:

- Lock-free structures eliminate the possibility of deadlocks since they don't rely on locks for synchronization.
- Lock-free algorithms allow multiple threads to make progress concurrently, improving performance in highly parallel environments.
- Lock contention in traditional locks can lead to frequent context switching. Lock-free approaches avoid this overhead.
- While not all threads are guaranteed to complete in a single step, lock-free structures ensure at least one thread always progresses, preventing complete stalls

Thread Libraries and Synchronization

Lock Free Data Structure: Working Principle

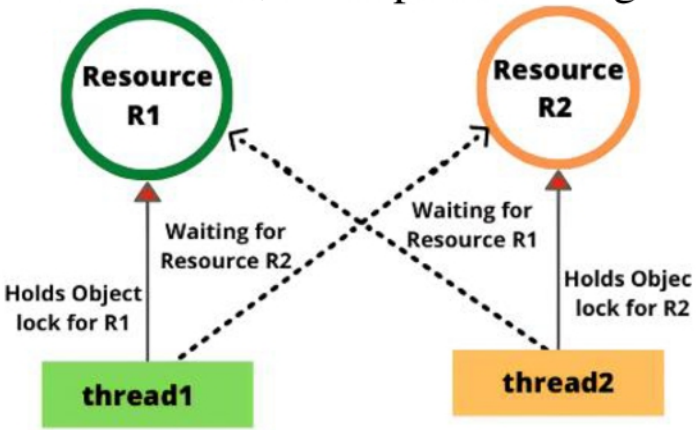
Disadvantage;

- Lock-free algorithms are much more complex to design and implement correctly compared to lock-based algorithms.
- Frequent retries due to conflicts (e.g., failed CAS operations) can degrade performance under high contention.
- Lock-free implementations often rely on platform-specific atomic instructions, making them less portable across architectures.
- Threads may repeatedly retry operations due to contention, leading to livelock scenarios where progress is slow or negligible.

Deadlock Detection Algorithm

Introduction

- A set of Threads is deadlock if each thread is waiting for an event that only another thread in the same set can cause.
- Because all the threads are waiting, none of them ever cause any of the events that could wake up any of the other member of the set, and all the threads continue to wait forever.
- Let us consider a system consists of two thread (t1 and t2) and two resources (R1 and R2). If both thread need both resources to complete their task, Example: Reading from scanner(R1) and Printing from Printer (R2), then two process will enter to a state called deadlock and wait forever if



i. Thread 1 Request Resource1 and is granted.

ii. Thread 2 request Resource2 and is granted.

iii. Thread 1 request Resource2 and is queued up, pending the release of Resource2 by Thread 2.

iv. Thread 2 request Resource1 and is queued up, pending the release of Resource1 by Process1.

Deadlock Detection Algorithm

Condition for Deadlock

- For a given set of process, we can say a deadlock is said to occur if all of the following four conditions occur simultaneously.

1. Mutual exclusion
   Only one process at a time can use a resource.
2. Hold and wait
   Process holding at least one resource is waiting to acquire additional resources held by other processes.
3. No preemption
   Resources are released only voluntarily by the process holding the resource, after the process is finished with it
4. Circular wait
   There exists a set {P1 , …, Pn } of waiting processes.
   P1 is waiting for a resource that is held by P2
   P2 is waiting for a resource that is held by P3
   …
   Pn is waiting for a resource that is held by P1

All of these four conditions must be present for a deadlock to occur. If one or more of these conditions is absent, no Deadlock is possible.

Deadlock Detection Algorithm

Condition for Deadlock

- For a given set of process, we can say a deadlock is said to occur if all of the following four conditions occur simultaneously.

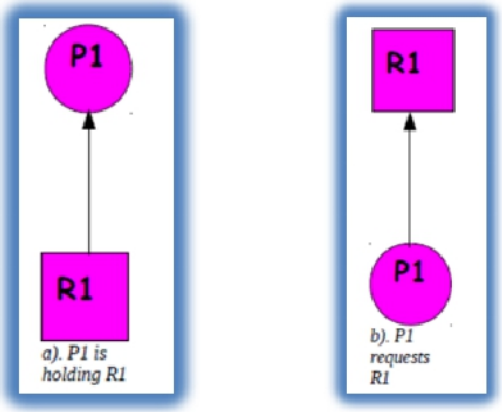
1. Mutual exclusion
   Only one process at a time can use a resource.
2. Hold and wait
   Process holding at least one resource is waiting to acquire additional resources held by other processes.
3. No preemption
   Resources are released only voluntarily by the process holding the resource, after the process is finished with it
4. Circular wait
   There exists a set {P1 , …, Pn } of waiting processes.
   P1 is waiting for a resource that is held by P2
   P2 is waiting for a resource that is held by P3
   …
   Pn is waiting for a resource that is held by P1

All of these four conditions must be present for a deadlock to occur. If one or more of these conditions is absent, no Deadlock is possible.

Deadlock Detection Algorithm

Modelling Deadlock

- Above four conditions can be modeled using graph called resource allocation graph or directed graph or wait for graph.
- In this graph, circle are used to represent process, and square are used to represent resource.
- An arc from square(Resources) to circle(Process) means that the resource previously requested has been granted and now holding by that process.
- Similarly, an arc from circle (Process) to square (Resource) means that the process is currently requesting and waiting for the resource.



Deadlock Detection Algorithm

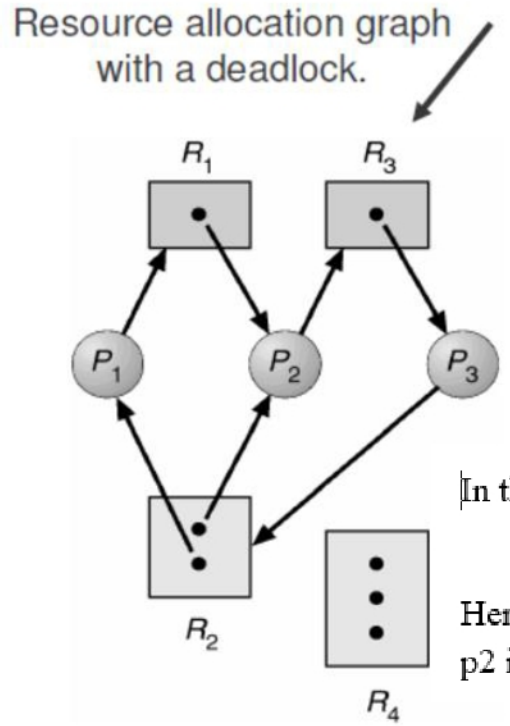
Modelling Deadlock

- If there is no cycle in a graph, then there is no deadlock but
- If there is cycle then,
    i. If each resource type has 1 instance, then deadlock is said to occur.
    ii. If resources type has multiple instance, then deadlock may or may not exist.

Deadlock Detection Algorithm

Example 1:

Three Processes P1, P2, P3 and Resource R2 having 2 instance, R4 having 3 instance, R1 and R3 having single instance with Resource Allocation as below



In the above graph, we see two cycles as below

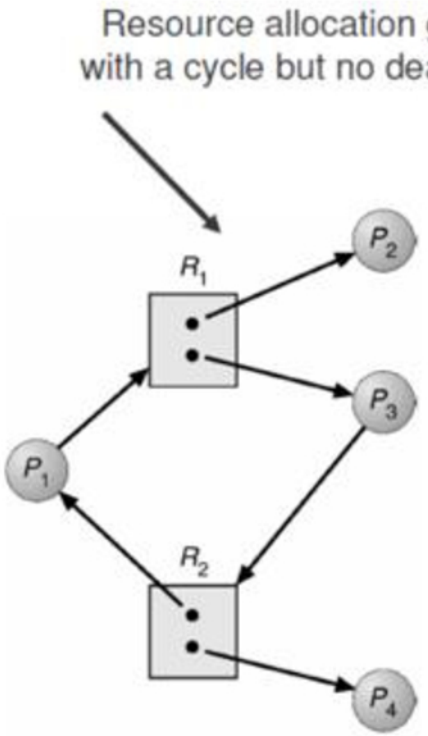
P1 → R1 → P2 → R3 → P3 → R2 → P1
P2 → R3 → P3 → R2 → P2

Here, process P1, P2 and P3 are deadlocked because P1 is waiting for resource R1 held by p2, p2 is waiting for resource R3 held by p3 and p3 is waiting for resource R2 held by p1and p2

Deadlock Detection Algorithm

Example 2:

Four Processes P1, P2, P3, P4 and Resource R1 having 2 instance, R2 having 2 instance with Resource Allocation as below



In the above graph, we see a cycle as below

$$P1 \rightarrow R1 \rightarrow P3 \rightarrow R2 \rightarrow P1$$

Despite of presence of cycle in the graph, we can't say the process P1 and P2 are deadlock because

i. After completing its task by process p4 it will release one instance of resource R2 hold by it, thus allowing the process P3 to acquire both resource i.e. one instance of R1 and one instance of R2 hence breaking the cycle.

ii. Or, when process p2 complete its task then it will release one instance of resource R1 hold by it, thus allowing the process P1 to acquire both resource i.e. one instance of resource R2 and one instance of resource R1 hence breaking the cycle.

Deadlock Detection Algorithm

Deadlock Detection and Recovery Approaches

- Deadlock detection is a technique which is used in to identify situations where processes or threads are in a circular wait, leading to a deadlock where none of the threads/processes can proceed.
- The detection and recovery algorithms aim to find such situations and take appropriate action to resolve them.
- There are three primary types of deadlock detection algorithms: **Centralized, Distributed, and Hierarchical**.

Synchronization in Distributed System

- Successful handover of computed data/result from one process to another is simply called Inter Process Communication.
- Different methods/techniques are used in IPCs as Monitor, Semaphore, message passing, and group communication.
- IPCs is an entire story and closely related to the process of cooperation and synchronization. From view of operating systems how are the critical regions are implemented and resources are allocated are major focus.
- In single CPU systems, critical regions, mutual exclusion and others synchronization problems are solved mostly by using semaphore, Monitors etc.
- But, these methods are only not suited for the distributed environment. In case of distributed operating system other techniques are also needed.

Synchronization in Distributed System

- Communication between processes in a distributed system can have unpredictable delays, processes can fail, messages may be lost .
- Synchronization in distributed systems is harder than in centralized systems because the need for distributed algorithms.
- Synchronization in distributed system is more complicated than in the centralized system because of 4 major reasons.
    - Information is scatter among multiple machine.
    - Processes make decision based on only on local information.
    - A Single point of failure in system should be avoided
    - No common clock or other precise global time source exists
- Points number 1 to 3 explain that it is unpredictable to collect all the information in a single place for processing because for resource allocation all requests need to send into a single manager, which examines and grant or denies request based on resource allocation table. For heavy system it is great burden on particular machine or process only. Moreover, if a system is failed then entire system become non predictable or unreliable
- Clock is crucial issue because it is ambiguous. If a process of a system wants to talk to its kernel. It means the time of asking and reply should hold the principles HAPPEN BEFORE.

Clock Synchronization

- In a distributed system the internal clocks of several computers may differ with each other .
- Even when initially set accurately, real clocks will differ after some time due to
  - clock drift(rate at which clock tick), caused by clocks counting time at slightly different rates.
- Clock Synchronization is the process of setting all the cooperating systems of distributed network to the same logical or physical clock.
- Clock synchronization is dramatic and it is fitting in process execution. But, Is it possible to synchronize all the clocks in the distributed environment into single clock. There are different concepts and implementations regarding the clock synchronization by using logical and physical clocks.
  1. **Physical clocks** - clocks whose values must not deviate from the real time by more than a certain amount.
  2. **Logical clocks** - to provide consistent event ordering

Clock Synchronization

Physical clocks Synchronization- Cristian's algorithm

- Cristian's algorithm relies on the existence of a time server as a physical clock synchronization algorithm.
- The time server maintains its clock by using a radio clock or other accurate time source, then all other computers in the system stay synchronized with it.
- A time client will maintain its clock by making a procedure call to the time server.
- Variations of this algorithm make more precise time calculations by factoring in network radio propagation time.
- The time server needs to change its time gradually. The time server, while responding to a client, needs to consider msg delays, subtract

(T1 - T0 – I) which is called Round Trip Time (RTT)

Clock Synchronization

Physical clocks Synchronization- Cristian's algorithm

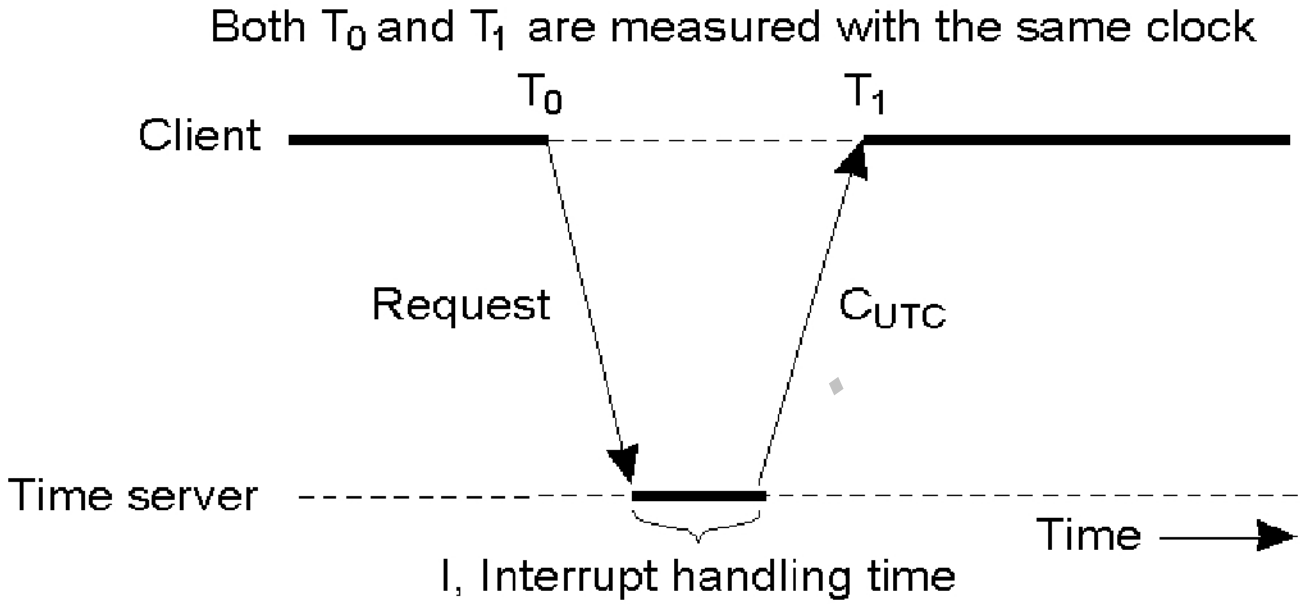


Fig: Getting Current Time of Server

Clock Synchronization

Physical clocks Synchronization- Cristian's algorithm

- Cristian's Algorithm works between a process P, and a time server S — connected to a source of UTC (Coordinated Universal Time. Then the algorithm goes like this.
  1. P requests the time from S
  2. After receiving the request from P, S prepares a response and appends the time T from its own clock.
  3. P then sets its time to be T + RTT/2
- P needs to record the Round Trip Time (RTT) of the request it made to S so that it can set its clock to T + RTT/2. This method assumes that the RTT is split equally between both request and response, which may not always be the case but is a reasonable assumption on a LAN connection.
- Further accuracy can be gained by making multiple requests to S and using the response with the shortest RTT. We can estimate the accuracy of the system as follows.
- Let min be the minimum time to transmit a message one-way(ideal condition, no delay). The earliest point at which S could have placed the time T, was min after P sent its request. Therefore, the time at S, when the message is received by P, is in the range (T + min) to (T + RTT - min). The width of this range is (RTT - 2*min). This gives an accuracy of (RTT/2 - min).

Clock Synchronization

Physical clocks Synchronization- Cristian's algorithm

- Send request at 5:08:15.100 (T_0)
- Receive response at 5:08:15.900 (T_1)

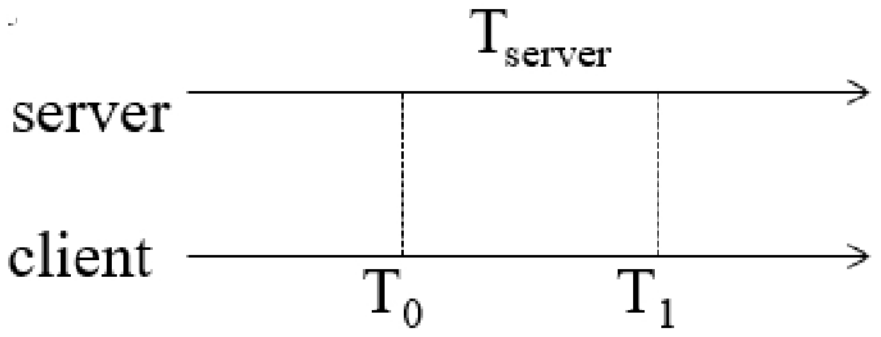
  Response contains 5:09:25.300 (T_{server})
- Round-trip time is $T_1 - T_0$

  5:08:15.900 - 5:08:15.100 = 800 ms
- Best guess: timestamp was generated 400 ms ago

Set the local time to T_{server} + round-trip-time/2

5:09:25.300 + 400 = 5:09.25.700

- Accuracy: ± round-trip-time/2



Clock Synchronization

Logical clocks Synchronization- Lamport's Algorithm

- Lamport invented a simple mechanism by which the happened-before ordering can be captured numerically.
- A Lamport logical clock is a incrementing software counter maintained in each process.
- Lamport's Consideration.
    - The happened-before relation is a partial ordering of events in distributed systems such that
    - If A and B are events in the same process, and A was executed before B, then

      A => B.

    - If A is the event of sending a message by one process and B is the event of receiving that by another process, then A => B.

      If A => B and B => C, then A => C.
- If two events A and B are not related by the this relation, then they are executed concurrently (no causal relationship)

Clock Synchronization

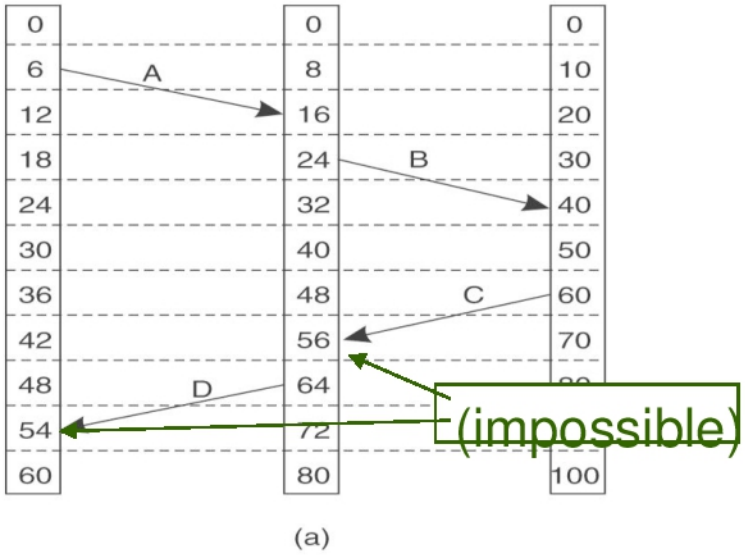
Logical clocks Synchronization- Lamport's Algorithm

- Lamport invented a simple mechanism by which the happened-before ordering can be captured numerically.
- A Lamport logical clock is a incrementing software counter maintained in each process.
- Algorithm follows:
    - A process increments its counter before each event in that process;
    - When a process sends a message, it includes its counter value with the message;
    - On receiving a message, the receiver process sets its counter to be greater than the maximum of its own value and the received value before it considers the message received. Further it is incremented by 1 unit.
- Conceptually, this logical clock can be thought of as a clock that only has meaning in relation to messages moving between processes. When a process receives a message, it resynchronizes its logical clock with that sender.
- With these actions the "happens-before" relationship of the message being sent and received is preserved.

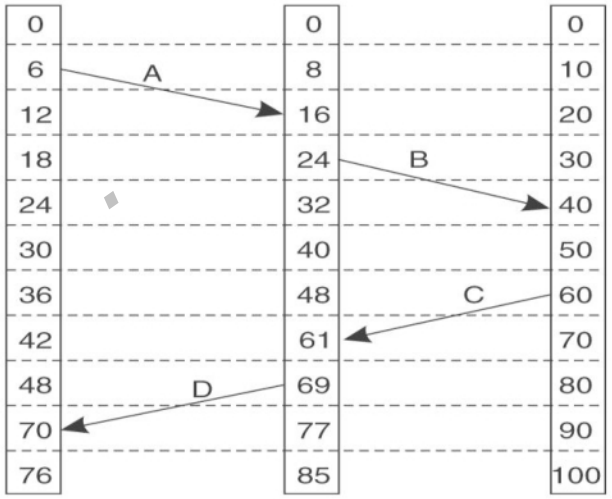
Clock Synchronization

Logical clocks Synchronization- Lamport's Algorithm

- Three processes, each with its own clock.
- The clocks run at different rates. Lamport's Algorithm corrects the clock.



(a)



(b)

- Note: ts(A) < ts(B) does not imply A happened before B.

Deadlock Detection Algorithm

Centralized Deadlock Detection

- A centralized system is one where all components of the system are controlled and managed by a single central entity, such as a central node or coordinator.
- Here one central node(coordinator) maintain global WFG and search for cycle.
- However, a single point of failure, Performance bottleneck of coordinator, Large Communication overhead for deadlock detection, are the demerits.

Distributed Deadlock Detection

- In a distributed deadlock detection system, there is no central coordinator.
- Instead, all processes in the system collaborate to detect deadlocks. The system relies on passing information between processes or nodes in the system to detect a circular wait.
- Each node is equally responsible in maintaining Global WFG and detecting deadlock.
- However, such systems are difficult to design, several sites may initiate detection for the same deadlock, proof of correctness is difficult, deadlock resolution is cumbersome.

Deadlock Detection Algorithm

Hierarchical Deadlock Detection

- The hierarchical deadlock detection algorithm(the most efficient and common approach) is a hybrid approach that combines elements of both centralized and distributed systems.
- The system is organized in a hierarchical manner, with a central coordinator for each group of processes (clusters), and each cluster may have its own coordinator for deadlock detection.
- Nodes organized in tree, where each site detects deadlocks involving only its descendants. Exploits access patterns local to a cluster of sites to efficiently detect deadlocks.

Deadlock Detection Algorithm

Centralized Deadlock Detection

The Ho-Ramamoorthy's Algorithm

- Each site maintains local WFG for its own processes and resources. While the central coordinator maintains the global WFG for the entire system(union of all local WFG).
- Now, the global wait-for graph may be constructed at different points at time:
    - Whenever a new edge is inserted or removed in one of the local wait-for graphs.
    - Periodically when a number of changes have occurred in a local wait-for graph.
    - Whenever the coordinator needs to invoke the cycle detection algorithm. The Central Control site periodically asks for the status tables.
- Control site analyzes global WFG and when it detects a cycle, a deadlock is detected.

Limitations:

- Phantom Deadlocks.(detection of false deadlock, a deadlock that doesn't really exist but is falsely detected)

Deadlock Detection Algorithm

Centralized Deadlock Detection

Phantom Deadlock

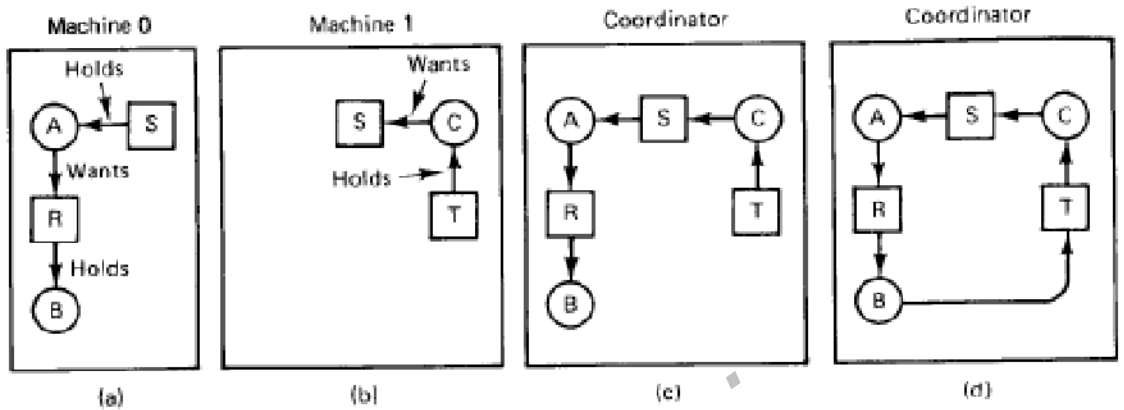


Fig. 3-23. (a) Initial resource graph for machine 0. (b) Initial resource graph for machine 1. (c) The coordinator's view of the world. (d) The situation after the delayed message.

- Suppose, B release R and asks for T, then two Message are sent to coordinator
    1. Message For releasing R
    2. Message indicating Waiting for T
- If Message 2 Arrives to the control site/coordinator first. Coordinator make global WFG with cycle , detecting a phantom deadlock.

Solution: Global Time stamp(Lamport Algorithm)

Deadlock Detection Algorithm

Centralized Deadlock Detection

Solution: Global Time stamp(Lamport Algorithm)

- Every Site (Here node B) maintains a logical clock.
- When B releases R and sends a message to the coordinator, it attaches a timestamp T_1 to the message.
- When B requests T and sends the message, it attaches a timestamp T_2, where $T_2 > T_1$.
- Coordinator Handles Messages Based on Timestamps:
- The coordinator maintains a queue of incoming messages and processes them in the order of their timestamps.
- Even if Message 2 (requesting T) arrives before Message 1 (releasing R), the coordinator waits until Message 1 (the message with the earlier timestamp) is processed.
- By ordering messages based on timestamps, the coordinator ensures that the global wait-for-graph (WFG) is built in the correct sequence.
- This prevents the detection of a cycle (deadlock) that doesn't exist in the actual system.

Deadlock Detection Algorithm

Distributed Deadlock Detection

Chandy-Misra-Haas Edge Chasing algorithm

- This Algorithm uses a probe message to detect the presence of cycles.
- The blocked process sends 'probe' message to the resource holding process
- Probe' message contains:
    i. ID of blocked process.
    ii. ID of process sending the message.
    iii. ID of process to which the message was sent.
- When the probe is received by the blocked process it forwards it to the processes holding the requested resources.
- If the blocked process receives it own probe it means that a cycle exists and hence the deadlock also exists.

Deadlock Detection Algorithm

Distributed Deadlock Detection

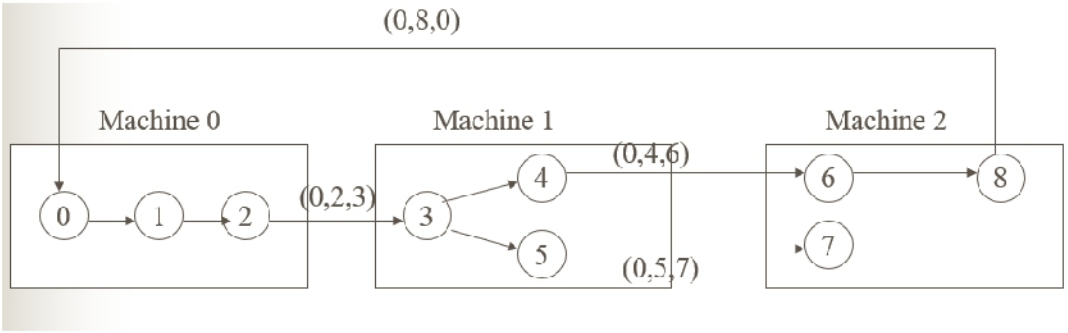
Chandy-Misra-Haas Edge Chasing algorithm

- This algorithm is a probe-based distributed algorithm for detecting deadlocks in systems using a distributed WFG.
- The algorithm sends probes (special messages) through the WFG to detect cycles. A cycle in the distributed WFG indicates a deadlock.
- **Algorithm**
  - When a process Pi, starts waiting for a resource held by any process Pj, Pi sends a probe message to Pj. A probe message contains three fields: [i, j, k] where:
    - i: Id of the blocked process i.e. the process that initiated the probe.
    - j: Id of the process that sending the probe.
    - k: Id of the process to which the probe is sent.
  - When Pj receives a probe [i,j,k]
    - If j=i (i.e., Prove returns to the initiating Process), this indicates a cycle (i.e., a deadlock). Pi declares a deadlock.
    - Otherwise, Pj forwards the probe to all processes Pk that it is waiting for.

Deadlock Detection Algorithm

Distributed Deadlock Detection

Chandy-Misra-Haas Edge Chasing algorithm



- The initial message from 0 to 1 contain (0,01), then from 1 to 2 will contain (0, 1 ,2)
- In the above diagram we see the remote messages labelled (0,2,3), (0,4,6), (0,5,7), (0,8,0)
- If a message goes all the way around and come back to the original sender, that is, the

Solution: process listed in the first field, a cycle exists and the system is in deadlock.

- Have process that initiated the probe commit suicide. Problem : if several processes invoke algorithm simultaneously. If each discover the deadlock, each would kill.
- Each process add its ID to the end of probe message so that when it returned to the initial sender, the complete cycle would be listed. Sender then can see the process with highest number(ID) and kill that one or send a message asking to kill itself.

Deadlock Detection Algorithm

Hierarchical Deadlock Detection

Menasce-Muntz Algorithm

- Sites(Controllers) are organized like tree in structure.
- Each **leaf node** maintains a Local Wait-For Graph (WFG).
- At the **leaf nodes**:
  - •Construct the Local WFG for processes and resources.
  - •Detect local cycles; if found, report to the parent node.
  - •Propagate external dependencies(Process waiting on remote resources) to the parent node.
- At the **parent nodes**:
  - Collect WFG updates from child nodes.
  - Merge updates into a Consolidated WFG.
  - Detect cycles in the Consolidated WFG.
- Propagate updated WFG to the next-level parent node, if applicable.
- At the **root node**:
  - Perform final cycle detection.
  - Report deadlock if detected or confirm deadlock-free status.
- For **deadlock resolution**:
  - Identify processes in the deadlock cycle.
  - Select processes to terminate or roll back to break the deadlock.
- Notify relevant nodes to take corrective action.

Distributed Mutual Exclusion Algorithms

There are two basic approaches to distributed mutual exclusion:

1) Non-token-based (Permission based):

- Each process freely and equally competes for the right to use the shared resource; requests are arbitrated by a central control site or by distributed agreement.
    - Permission From Central Coordinator: Central Coordinator Based Algorithm
    - Permission from All(Distributed Process): Lamport Algorithm, Ricart Agarwala Algorithm
    - Permission from Subset: Maekawa Algorithm

2) Token-based:

- A logical token representing the access right to the shared resource is passed in a regulated fashion among the processes; whoever holds the token is allowed to enter the critical section.
    - Token Ring Algorithm
    - Suzuki Kasami Algorithm

Distributed Mutual Exclusion Algorithms

Central Coordinator Based Algorithm

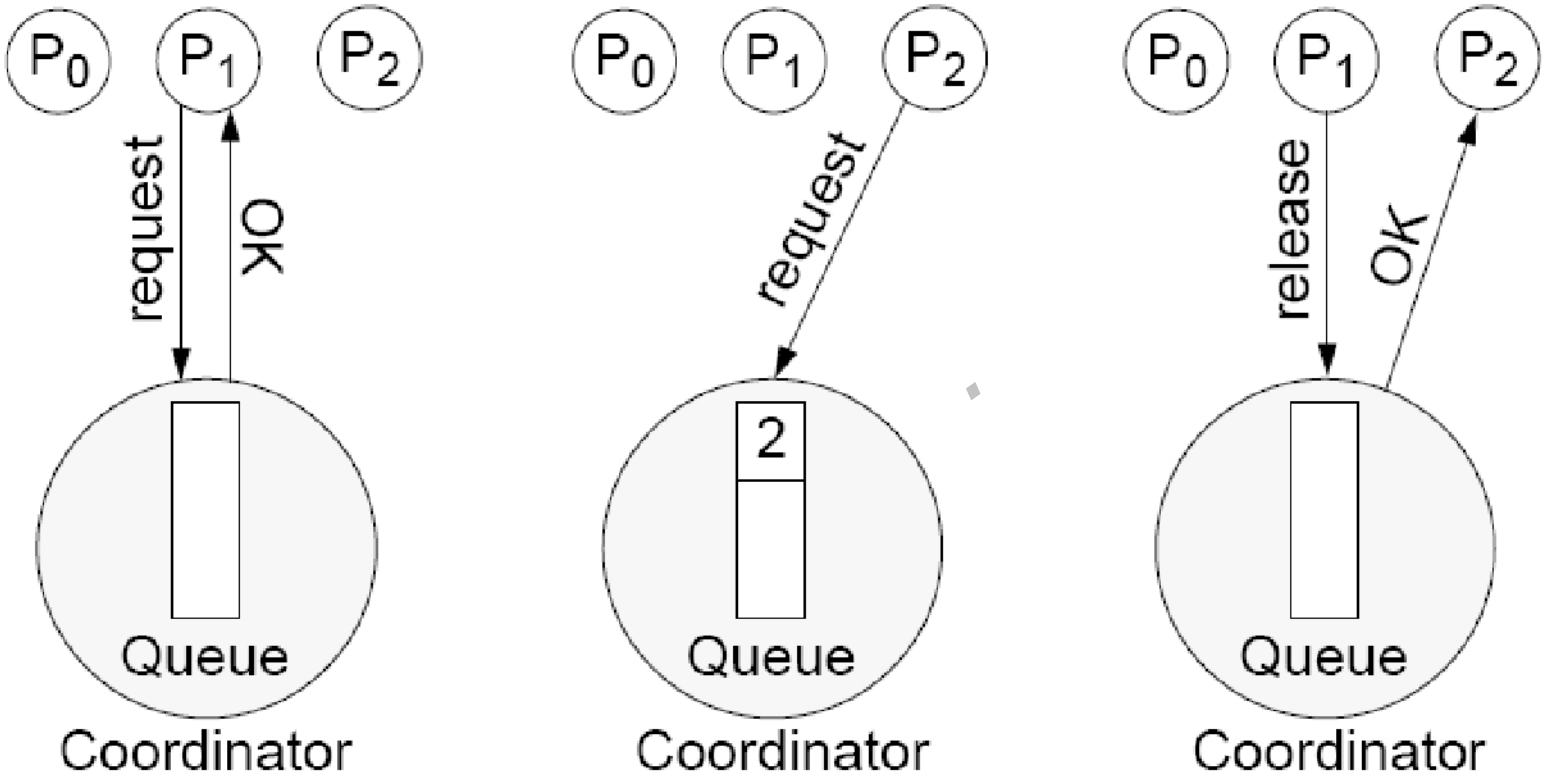
- A central coordinator grants permission to enter a CS
- In this algorithm, a process that wants to enter into a CS has to take a permission from a central coordinator
- The scheme is simple and easy to implement.
- The strategy requires only three messages per use of a CS (*request, OK, release).*

Algorithm

- To enter a CS, a process sends a request message to the coordinator and then waits for a reply (during this waiting period the process can continue with other work).
- The reply from the coordinator gives the right to enter the CS.
- After finishing work in the CS the process notifies the coordinator with a release message.

Distributed Mutual Exclusion Algorithms

Central Coordinator Based Algorithm



Distributed Mutual Exclusion Algorithms

Central Coordinator Based Algorithm: Pseudocode

```
Coordinator
     loop
       receive(msg);
         case msg of
           REQUEST: if nobody in CS
                    then reply GRANTED
                    else queue the REQ;
                         reply DENIED
           RELEASE: if queue not empty then
                     remove 1st on the queue
                     reply GRANTED
   //modified algo
         end case
     end loop

Client
     send(REQUEST);
     receive(msg);
     while msg != GRANTED then receive(msg);
     enter CS;
     send(RELEASE)
```

Distributed Mutual Exclusion Algorithms

Central Coordinator Based Algorithm: Pseudocode

Advantage

- Guarantees mutual exclusion
- It is fair (First Come First Served
- Easy to implement

Disadvantages

- The coordinator can become a performance bottleneck.
- The coordinator is a critical point of failure:
    - *If the coordinator crashes, a new coordinator must be created. The coordinator can be one of the processes competing for access;* ***an election algorithm*** *has to be run in order to choose one and only one new coordinator.*
    - *If no explicit DENIED message, then cannot distinguish permission denied from a dead coordinator*

Distributed Mutual Exclusion Algorithms

Permission from all (distributed) Processes

- In a distributed environment it seems more natural to implement mutual exclusion, based upon distributed agreement - not on a central coordinator.
- The basic algorithm goes like this:
    - When any process (say P1, P2) wants to enter a critical section, they sends a request message to all other processes in the system (P2, P3, ..., Pn)
    - All events are time-stamped by the **global ordering algorithm**
    - When a process (Say p3) receives a request from any process (say P1), it compares timestamps and delays its acknowledgment if it is in the critical section or has its own request, responding only after completing its critical section, otherwise responding immediately to the request message.
    - If multiple request available, then, the process whose request message has the smallest timestamp is granted permission to enter the critical section first, while other waits in the request queue.
    - Once a process completes its critical section, it broadcasts a release message to all other processes, indicating that it has finished and they can now proceed with their requests.

Distributed Mutual Exclusion Algorithms

Permission from all (distributed) Processes

- **Two distributed algorithms**
    1. Lamport's Algorithm
    2. Ricart Agarwala Algorithm

Distributed Mutual Exclusion Algorithms

Lamport's Algorithm

- Lamport's Distributed Mutual Exclusion Algorithm is a permission based algorithm proposed by Lampor.
- Here timestamp is used to order critical section requests and to resolve any conflict between requests.
- In Lamport's Algorithm critical section requests are executed in the increasing order of timestamps i.e a request with smaller timestamp will be given permission to execute critical section first than a request with larger timestamp.
- In this algorithm:
  - Three type of messages (REQUEST, REPLY and RELEASE) are used and communication channels are assumed to follow FIFO order.
  - A site send a REQUEST message to all other site to get their permission to enter critical section.
  - A site send a REPLY message to requesting site to give its permission to enter the critical section.
  - A site send a RELEASE message to all other site upon exiting the critical section.

Distributed Mutual Exclusion Algorithms

Lamport's Algorithm

- Every site Si, keeps a queue to store critical section requests ordered by their timestamps. $request_queue_i$ denotes the queue of site Si
- A timestamp is given to each critical section request using Lamport's logical clock.

Algorithm:

- **To enter Critical section:**
  - When a site S_i wants to enter the critical section, it sends a request message **Request(ts_i, i)** to all other sites and places the request on **$request_queue_i$**. Here, Ts_i denotes the timestamp of Site S_i
  - When a site S_j receives the request message **REQUEST(ts_i, i)** from site S_i, it returns a timestamped REPLY message to site S_i and places the request of site S_i on **$request_queue_j$**
- **To execute the critical section:**
  - A site S_i can enter the critical section if it has received the message with timestamp larger than **(ts_i, i)** from all other sites and its own request is at the top of **$request_queue_i$**
- **To release the critical section:**
  - When a site S_i exits the critical section, it removes its own request from the top of its request queue and sends a timestamped **RELEASE** message to all other sites
  - When a site S_j receives the timestamped **RELEASE** message from site S_i, it removes the request of S_i from its request queue

Distributed Mutual Exclusion Algorithms

Lamport's Algorithm

Message Complexity: Lamport's Algorithm requires invocation of 3(N – 1) messages per critical section execution. The 3(N – 1) messages involves

- (N – 1) request messages
- (N – 1) reply messages
- (N – 1) release messages

Advantages of Lamport's Algorithm for Mutual Exclusion in Distributed System:

1. **Simplicity:** Lamport's algorithm is relatively easy to understand and implement compared to other algorithms for mutual exclusion in distributed systems.
2. **Fairness:** The algorithm guarantees fairness by providing a total order of events that is used to determine the next process that can enter the critical section.
3. **Scalability:** Lamport's algorithm is scalable because it only requires each process to communicate with its neighbors, rather than all other processes in the system.
4. **Compatibility:** The algorithm is compatible with a wide range of distributed systems and can be adapted to different network topologies and communication protocols.

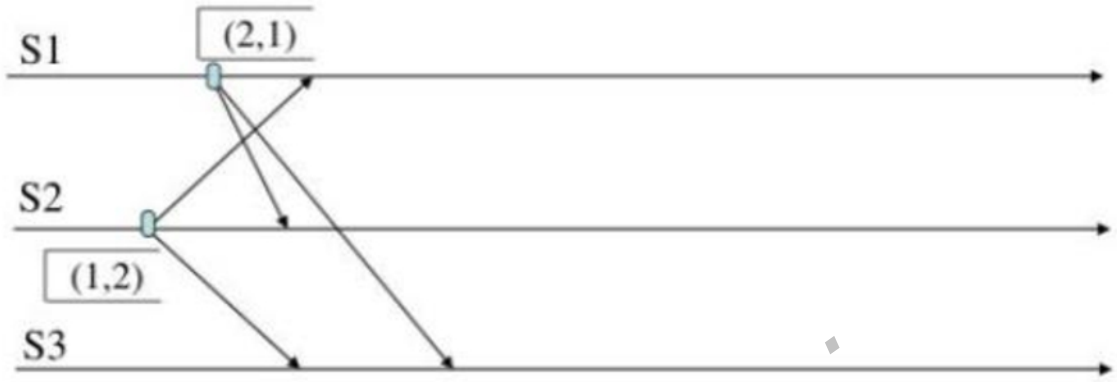
Disadvantages of Lamport's Algorithm for Mutual Exclusion in Distributed System:

1. **Message Overhead:** The algorithm requires a lot of message passing between processes to maintain the total order of events, which can lead to increased network traffic and latency.
2. **Delayed Execution:** The algorithm can lead to delays in the execution of critical sections because a process may have to wait for messages to arrive from other processes before entering the critical section.
3. **Limited Performance:** The algorithm may not perform well in systems with a high number of processes because of the increased message overhead and delayed execution.
4. **No Fault Tolerance:** The algorithm does not provide any fault tolerance mechanism, which means that if a process fails or crashes, it may cause the entire system to fail or become unstable.

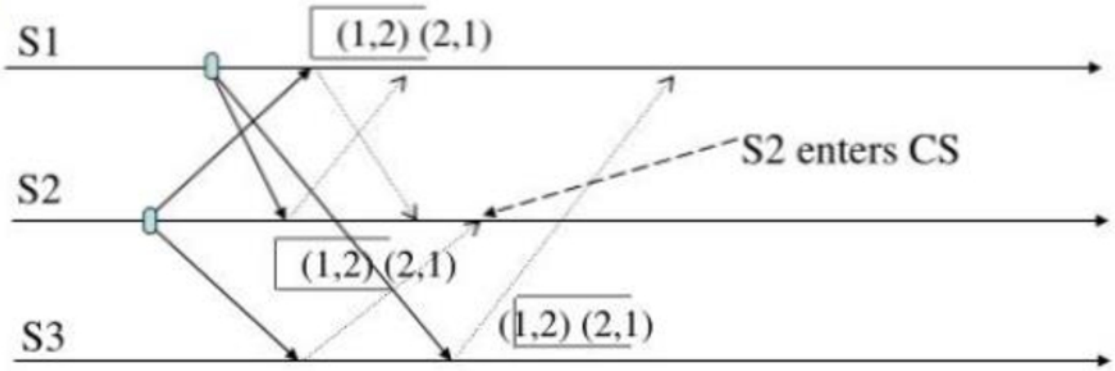
Distributed Mutual Exclusion Algorithms

Lamport's Algorithm: Example

Step 1:

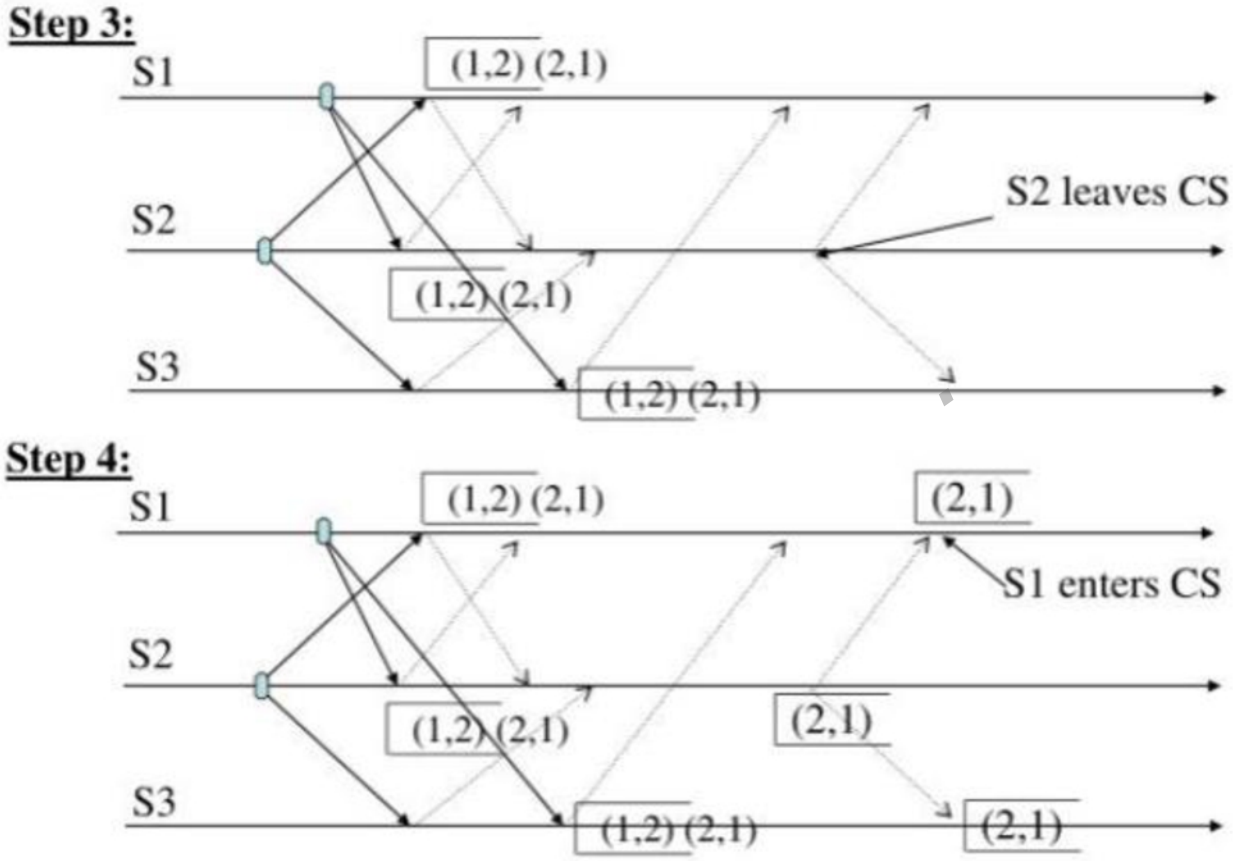


Step 2:



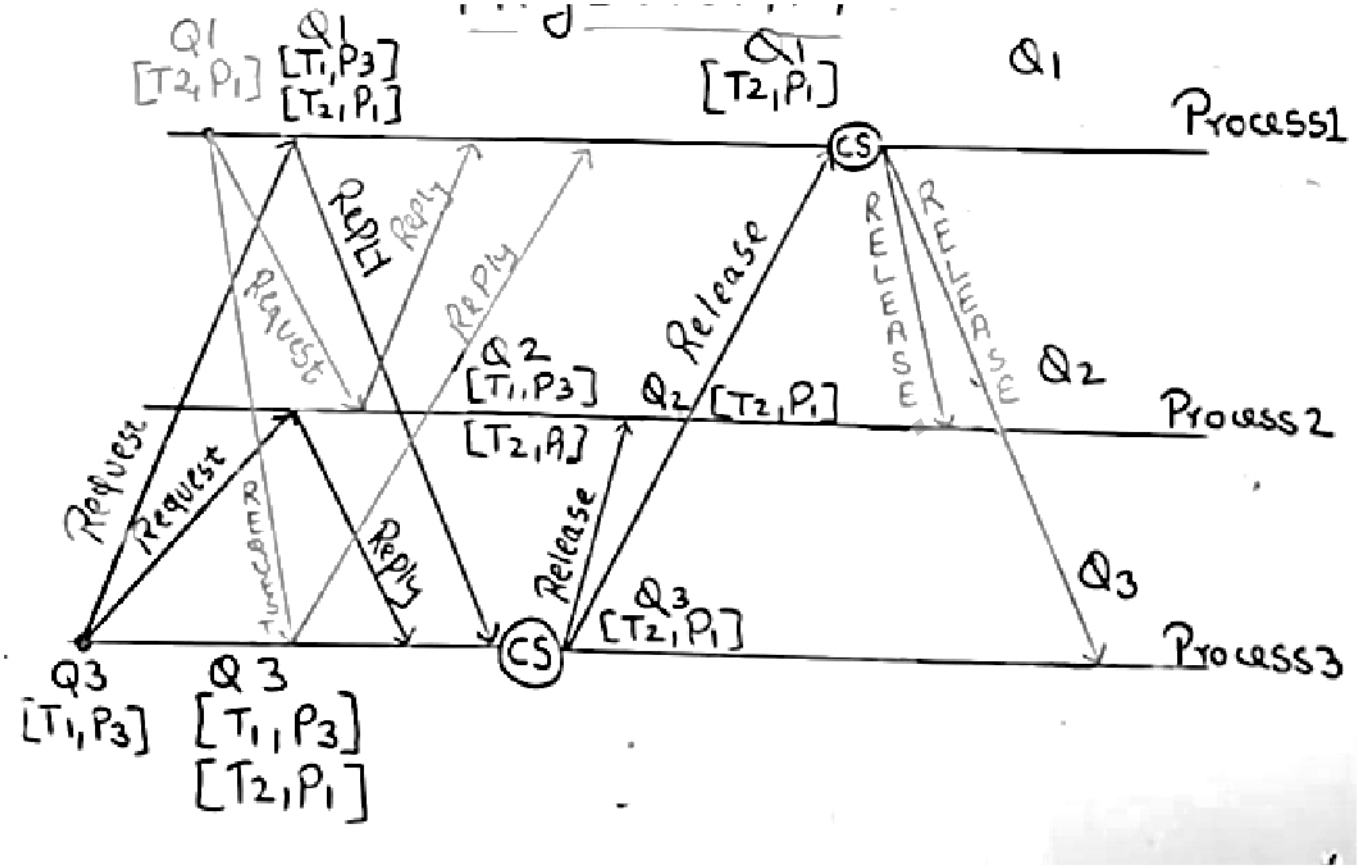
Distributed Mutual Exclusion Algorithms

Lamport's Algorithm: Example



Distributed Mutual Exclusion Algorithms

Example: 2



Distributed Mutual Exclusion Algorithms

Ricart Agarwala Algorithm

- It is an algorithm for mutual exclusion in a distributed system proposed by Glenn Ricart and Ashok Agrawala.
- It is an extension and optimization of Lamport's Distributed Mutual Exclusion Algorithm.
- Two type of messages (REQUEST and REPLY) are used and communication channels are assumed to follow FIFO order.
- A site send a REQUEST message to all other site to get their permission to enter the critical section.
- A site send a REPLY message to another site to give its permission to enter the critical section.
- A timestamp is given to each critical section request using Lamport's logical clock.
- Smaller timestamp gets high priority over larger timestamp.

Distributed Mutual Exclusion Algorithms

Ricart Agarwala Algorithm

Algorithm:

- **To enter Critical section:**
  - When a site S_i wants to enter the critical section, it send a timestamped **REQUEST** message to all other sites.
  - When a site S_j receives a **REQUEST** message from site S_i, It sends a **REPLY** message to site S_i if and only if
    - Site S_j is neither requesting nor currently executing the critical section.
    - In case Site S_j is requesting, the timestamp of Site S_i's request is smaller than its own request.
- **To execute the critical section:**
  - Site S_i enters the critical section if it has received the **REPLY** message from all other sites.
- **To release the critical section:**
  - Upon exiting site S_i sends **REPLY** message to all the deferred requests.

Message Complexity: Ricart–Agrawala algorithm requires invocation of $2(N - 1)$ messages per critical section execution. These $2(N - 1)$ messages involves

- $(N - 1)$ request messages
- $(N - 1)$ reply messages

Distributed Mutual Exclusion Algorithms

Ricart Agarwala Algorithm

Advantage

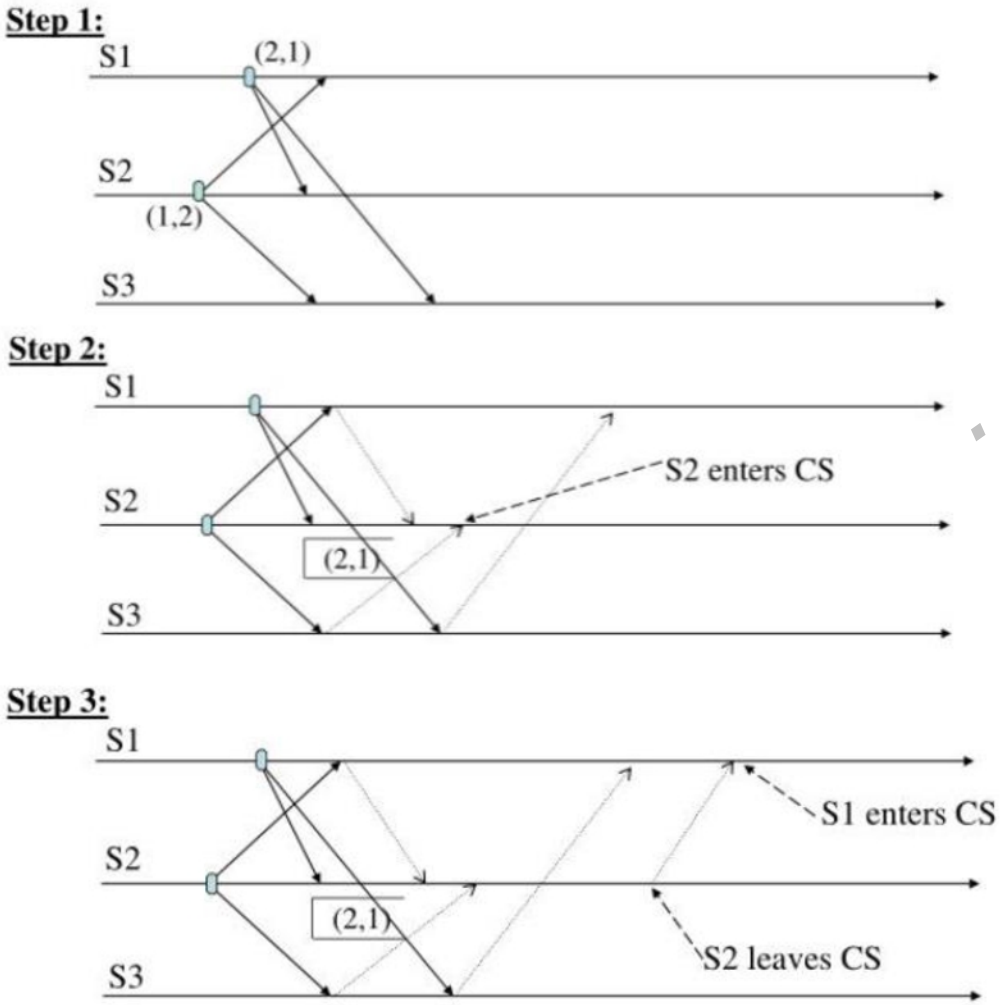
- **Low message complexity:** The algorithm has a low message complexity as it requires only (N-1) messages to enter the critical section, where N is the total number of nodes in the system.
- **Scalability:** The algorithm is scalable and can be used in systems with a large number of nodes.
- **Non-blocking:** The algorithm is non-blocking, which means that a node can continue executing its normal operations while waiting to enter the critical section.

Drawbacks of Ricart–Agrawala algorithm:

- **Unreliable approach**: failure of any one of node in the system can halt the progress of the system. In this situation, the process will starve forever. The problem of failure of node can be solved by detecting failure after some timeout.

Distributed Mutual Exclusion Algorithms

Ricart Agarwala Algorithm: Example



Distributed Mutual Exclusion Algorithms

Example 2:

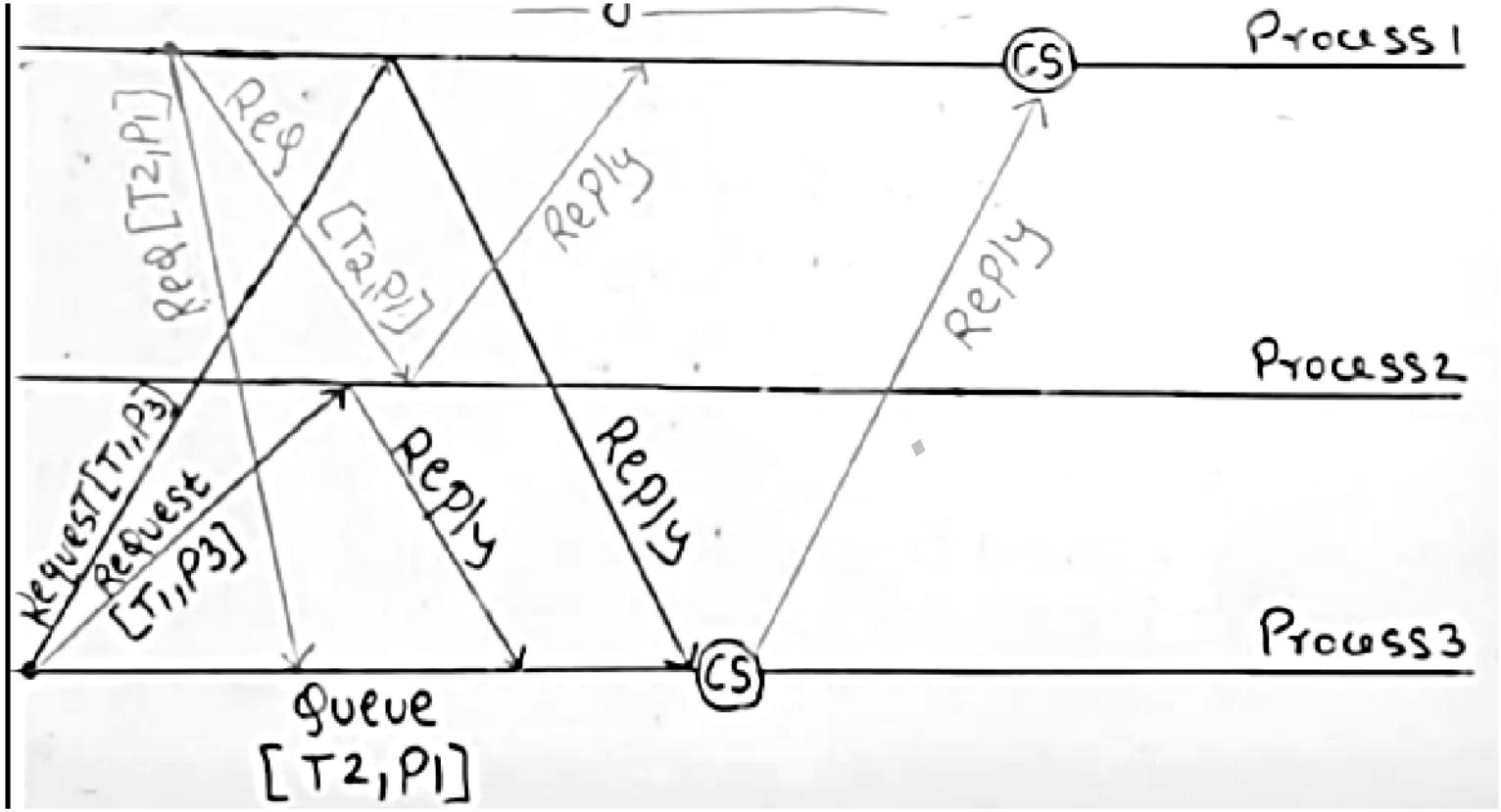


Fig: Ricart Agarwala Algorithm

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm

- It is quorum (subset of nodes) based approach to ensure mutual exclusion in distributed systems.
- The algorithm ensures that only one process at a time can execute in the critical section, preventing race conditions in distributed systems.
- Instead of requiring all nodes to participate in granting permission (as in Ricart-Agrawala), it divides the nodes into quorums, where only a subset of nodes need to agree.
- In this algorithm:
    - Three type of messages (REQUEST, REPLY and RELEASE) are used.
    - To **request Critical Section**, A site (S_i) send a REQUEST message to all other site in its request set(R_i) or quorum to get their permission to enter critical section.
    - Upon **receiving a REQUEST message**, a site send a REPLY message(if no REPLY message has been sent since the last RELEASE message is received) to requesting site to give its permission to enter the critical section. Also, status is updated, to indicate REPLY has been sent. Queue is maintained if any site is already inside the critical section.

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm

- For existing from critical section, A site send a RELEASE message to all other site in its request set R_i or quorum upon exiting the critical section.
- To enter critical section, site (S_i) can enter critical section after receiving REPLY message from all node in R_i
- Upon receiving a release message, processes check their queue to see if another process can now enter the critical section based on the ordering (timestamps and process IDs).
- The process with the smallest timestamp and process ID is allowed to enter the critical section next

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm

- A request set or Quorum in Maekawa's algorithm must satisfy the following properties:.

$$\forall i\ \forall j : i \neq j,\ 1 \leq i, j \leq N :: R_i \cap R_j \neq \emptyset$$

- **i.e** there is at least one common site between the request sets of any two sites.

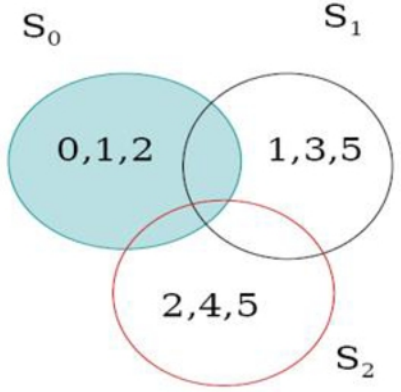
$$\forall i : 1 \leq i \leq N :: S_i \in R_i$$

$$\forall i : 1 \leq i \leq N :: |R_i| = K \text{ i.e. every request set contain K members}$$

- Any site S_i is contained in exactly K sets. (K is size of quorem.)

$$N = K(K - 1) + 1 \text{ and } |R_i| = \sqrt{N}$$

N represents the total number of sites (or nodes) in the distributed system.-



Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6,7,8,9

$S_0 = \{0, 1, 2\}$
$S_1 = \{1, 3, 5\}$
$S_2 = \{2, 4, 5\}$
$S_3 = \{0, 3, 4\}$
$S_4 = \{1, 4, 6\}$
$S_5 = \{0, 5, 6\}$
$S_6 = \{2, 3, 6\}$

Version 1 {Life of process I}

- A process that wants to enter the critical section sends a timestamped request to every other process in **Si**.
- When a process receives a request, it checks the timestamp of the request: If **this request has the lowest timestamp** among all pending requests (including its own), it sends an **ack** (acknowledgment) to the requesting process. If **another request has a lower timestamp**, it does **not send an ack** immediately. Instead, it queues the request and keeps it waiting.
- Once a process sends an **ack**, it commits itself (or "locks") to that requesting process.
- This ensures it won't grant access to the critical section to any other process until the current process releases it.

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6,7,8,9

$S_0 = \{0, 1, 2\}$
$S_1 = \{1, 3, 5\}$
$S_2 = \{2, 4, 5\}$
$S_3 = \{0, 3, 4\}$
$S_4 = \{1, 4, 6\}$
$S_5 = \{0, 5, 6\}$
$S_6 = \{2, 3, 6\}$

Version 1 {Life of process I}

- The requesting process can enter the critical section **only after receiving an acknowledgment (ack)** from every other process in **Si**. This ensures mutual exclusion
- After completing its work in the critical section, the process sends a **release** message to every process in **Si**.
- When a process receives a **release** message: It **unlocks itself**, meaning it can now grant access to other processes.
- It checks its queue for pending requests and sends an **ack** to the process with the **next lowest timestamp**.

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6,7,8,9

Version 1 {Life of process I}

$S_0 = \{0, 1, 2\}$

$S_1 = \{1, 3, 5\}$

$S_2 = \{2, 4, 5\}$

$S_3 = \{0, 3, 4\}$

$S_4 = \{1, 4, 6\}$

$S_5 = \{0, 5, 6\}$

$S_6 = \{2, 3, 6\}$

Let **i** and **j** attempt to enter their Critical Sections

$S_i \cap S_j \neq \emptyset$ implies there is a process **k** $\in S_i \cap S_j$

Process **k** will **never** send ack to both.

So it will act as the arbitrator and establishes ME

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6

$S_0 = \{0, 1, 2\}$
$S_1 = \{1, 3, 5\}$
$S_2 = \{2, 4, 5\}$
$S_3 = \{0, 3, 4\}$
$S_4 = \{1, 4, 6\}$
$S_5 = \{0, 5, 6\}$
$S_6 = \{2, 3, 6\}$

Deadlock Issue

- Assume three process 0, 1, 2 associated with subset S_i which want to enter their critical sections.
- Each process sends a **timestamped request** to every process in its subset Si.
- Each process in the subset Si sends an **acknowledgment (ack)** to one process, based on the timestamp priority.
- From S_0= {0,1,2}, 0,2 send *ack* to 0, but 1 sends *ack* to 1; i.e. process 0 receives ack from 2 and itself and waiting for release from process 1.
- From S_1= {1,3,5}, 1,3 send *ack* to 1, but 5 sends *ack* to 2;
- From S_2= {2,4,5}, 4,5 send *ack* to 2, but 2 sends *ack* to 0;

Now, 0 waits for 1 (to send a release), 1 waits for 2 (to send a release), , and 2 waits for 0 (to send a release), . So deadlock is possible!

Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6,7,8,9

$S_0 = \{0, 1, 2\}$
$S_1 = \{1, 3, 5\}$
$S_2 = \{2, 4, 5\}$
$S_3 = \{0, 3, 4\}$
$S_4 = \{1, 4, 6\}$
$S_5 = \{0, 5, 6\}$
$S_6 = \{2, 3, 6\}$

Avoiding Deadlock

- If processes always receive messages in increasing order of timestamp, then deadlock "could be" avoided.
- Send ***ack*** and set **lock** as usual.
- If **lock is set** and a request with a larger timestamp arrives, send ***failed*** *(you have no chance)*. If the incoming request has a lower timestamp, then send ***inquire*** *(are you in Critical Section?) to the locked process.*
- Receive ***inquire*** and at least one ***failed*** message → send ***relinquish***. The recipient resets the lock.

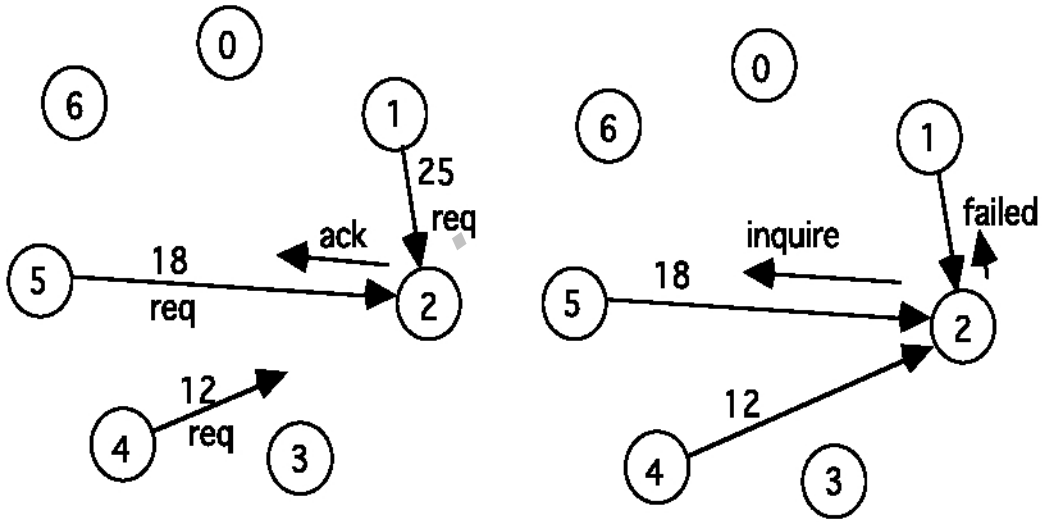
Distributed Mutual Exclusion Algorithms

Permission from Subset: Maekawa Algorithm, Example

Let there be seven processes 0, 1, 2, 3, 4, 5, 6,7,8,9

$S_0 = \{0, 1, 2\}$
$S_1 = \{1, 3, 5\}$
$S_2 = \{2, 4, 5\}$
$S_3 = \{0, 3, 4\}$
$S_4 = \{1, 4, 6\}$
$S_5 = \{0, 5, 6\}$
$S_6 = \{2, 3, 6\}$

Avoiding Deadlock



Distributed Mutual Exclusion Algorithms

Token Based Algorithm

- A token is circulated in a logical ring.
- A process enters its CS if it has the token.
- Issues:
  - If the token is lost, it needs to be regenerated.
  - Detection of the lost token is difficult since there is no bound on how long a process should wait for the token.
  - If a process can fail, it needs to be detected and then by-passed.
  - When nobody wants to enter, processes keep on exchanging messages to circulate the token

Distributed Mutual Exclusion Algorithms

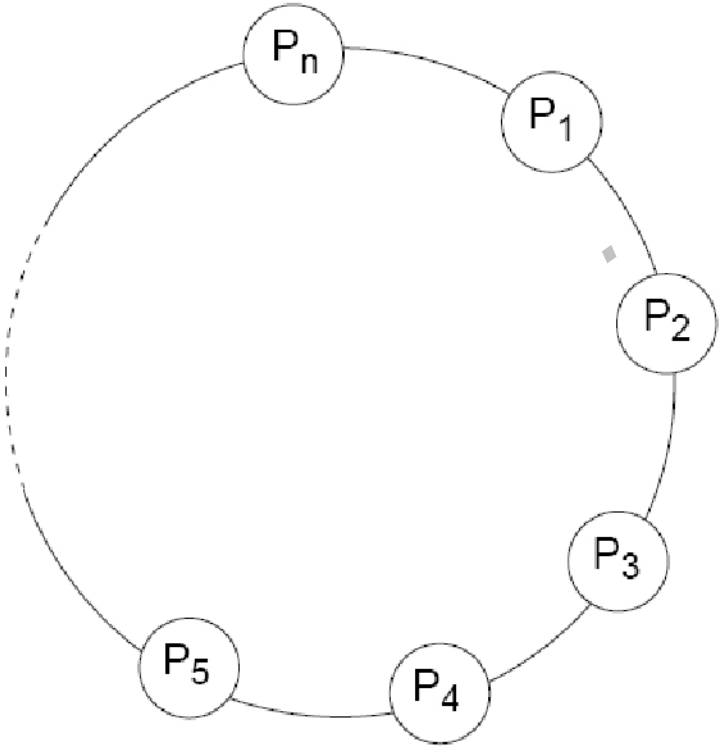
Token Based Algorithm

Token Ring Algorithm

- A very simple way to solve mutual exclusion ⇒ arrange the *n* processes P_1, P_2, ... P_n in a logical ring.
- The logical ring topology is created by giving each process the address of one other process which is its neighbour in the clockwise direction.
- The logical ring topology is unrelated to the physical interconnections between the computers.

Distributed Mutual Exclusion Algorithms

Token Based Algorithm: Token Ring Algorithm



Distributed Mutual Exclusion Algorithms

Token Based Algorithm: Token Ring Algorithm

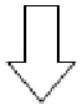
The algorithm

- The token is initially given to one process.
- The token is passed from one process to its neighbour round the ring.
- When a process requires to enter the CS, it waits until it receives the token from its left neighbour and then it retains it; after it got the token it enters the CS; after it left the CS it passes the token to its neighbour in clockwise direction.
- When a process receives the token but does not require to enter the critical section, it immediately passes the token over along the ring.

Distributed Mutual Exclusion Algorithms

Token Based Algorithm: Token Ring Algorithm

- It can take from 1 to *n*-1 messages to obtain a token. Messages are sent around the ring even when no process requires the token ⇒ additional load on the network.



The algorithm works well in heavily loaded situations, when there is a high probability that the process which gets the token wants to enter the CS. It works poorly in lightly loaded cases.

- If a process fails, no progress can be made until a reconfiguration is applied to extract the process from the ring.

- If the process holding the token fails, a unique process has to be picked, which will regenerate the token and pass it along the ring; an *election algorithm* (see later) has to be run for this purpose.

Further Study

- **Token Based Algorithm: Suzuki Kasami Algorithm**
- **Case Studies: Process Management in Linux and Windows.**